

Миамбо Жуниор Рикардо Агоштиньо

Тема ВКР: Разработка отказоустойчивой микросервисной архитектуры для мобильного приложения по оптимизации продуктивности с интеграцией машинного обучения.

Задание на ВКР:

Проектирование системы

- Разделение на доменные области: Проектирование структуры независимых микросервисов для четырех основных модулей: задачи (Task Manager Service), заметки (Note Service), таймер (Pomodoro Service) и дневника (Journal Service).
- Разработка стратегии синхронизации данных: Проектирование механизмов Offline-First, позволяющих мобильному приложению полноценно работать локально и синхронизировать изменения с бэкендом при восстановлении сети.

Разработка микросервисов

- Межсервисное взаимодействие: Реализация обмена данными между сервисами через высокопроизводительный протокол gRPC (для внутренних вызовов) и брокер сообщений RabbitMQ (для асинхронной передачи логов активности в ML-модуль).

Интеграция машинного обучения (Integration of ML)

- Сбор и подготовка данных: Поиск дата и адаптация открытых датасетов (Kaggle, GitHub) по паттернам продуктивности, тайм-менеджменту и анализу текстов для обучения специализированных моделей.
- Разработка модели МО: Обучение отдельных модулей для каждого микросервиса, Создание выделенного бэкенд-сервиса, который инкапсулирует обученные модели и предоставляет API для генерации общего анализа продуктивности.

Разработка мобильного приложения (Mobile Development)

- Реализация интерфейса: Разработка интерфейса для каждого модуля.
- Интеграция с ML-бэкендом: Создание логики взаимодействия с микросервисами.
- Реализация встроенной БД: Использование локального хранилища для мгновенного сохранения всех действий пользователя без ожидания ответа от сервера.
- Синхронизация состояний: Разработка механизма отслеживания изменений, который помечает новые или измененные локальные записи для последующей отправки на бэкенд.
- Кэширование прогнозов: Мобильное приложение должно сохранять последние полученные от ML-микросервиса отчеты и рекомендации, чтобы они оставались доступными пользователю даже без сети.

Аннотация:

Цель. Проектирование и реализация отказоустойчивой микросервисной платформы, объединяющей основные инструменты управления личной продуктивностью (управление задачами, трекинг привычек, таймер Помодоро и рефлексивный дневник) в единую экосистему с ИИ-аналитикой, обеспечивающей кросс-доменный анализ поведения пользователя на основе полного контекста его продуктивности.

Задачи.

1. Разработать микросервисную архитектуру: определить границы сервисов, протоколы взаимодействия, модель аутентификации и правила владения данными для системы независимо развёртываемых и масштабируемых сервисов.
2. Реализовать основные сервисы продуктивности: сервис self-manager (управление задачами, привычками и рутинами) и сервис timer (отслеживание сессий Помодоро).

3. Реализовать конвейер ИИ-аналитики по паттерну RAG: оркестрация сбора данных из всех сервисов, формирование структурированных промптов и вызов LLM-инференса.
4. Реализовать инфраструктуру кредитов и инференса с поддержкой трёх LLM-провайдеров (Ollama, Anthropic Claude, OpenAI GPT) и пути вывода sklearn-моделей.
5. Разработать веб-дашборды администратора и пользователя на основе Streamlit для управления системой и доступа к аналитике.
6. Реализовать offline-first синхронизацию с идемпотентным пакетным протоколом для мобильных клиентов на базе Kotlin Multiplatform.
7. Провести сквозную валидацию системы: верифицировать функциональную корректность, качество ИИ-анализа и операционную надёжность.

Результаты:

Разработана и развёрнута рабочая микросервисная платформа ProductiveSocial, состоящая из четырёх независимых сервисов (selfmanager, timer, analytics, billing) на Kotlin/Ktor и Python/FastAPI с отдельными PostgreSQL-базами данных на Neon и контейнеризацией через Docker/Colima. Реализована межсервисная аутентификация на основе разделяемого JWT-секрета и механизм UserRegistry для обеспечения канонической идентификации пользователя без централизованного провайдера.

Конвейер ИИ-аналитики реализован по паттерну RAG: сервис аналитики агрегирует полный контекст продуктивности пользователя из всех сервисов и передаёт его LLM через шесть специализированных типов анализа. По результатам сравнительного оценивания средняя оценка LLM-анализа составила 4,1/5 против 1,3/5 у базовой линии на основе правил по критериям релевантности, специфичности и действенности.

Проведено 12 задокументированных экспериментов, подтвердивших: корректность JWT-аутентификации между сервисами, математическую точность транзакционного учёта кредитов, полноту и идемпотентность offline-

синхронизации (100% записей без дубликатов), а также корректную деградацию при недоступности отдельных сервисов. Задержка холодного старта JVM-сервисов на бесплатном тарифе Render составила ~52 секунды; в прогретом состоянии сквозная задержка анализа — 15–30 секунд (с Ollama через ngrok). Архитектура спроектирована с явными точками расширения для добавления социального модуля, модуля дневника и проприетарных ML-моделей без структурных изменений существующих сервисов.

Содержание

- Введение
 - Актуальность проблемы
 - Значимость и миссия проекта
 - Цели и задачи
 - Новизна
 - Подход к валидации
 - Критерии успеха
- Глава 1: Анализ предметной области и видение решения
 - 1.1 Анализ предметной области
 - 1.2 Анализ проблемы "КАК ЕСТЬ"
 - 1.3 Обзор существующих решений
 - 1.4 Видение решения
 - 1.5 Определение приоритетов
- Глава 2: Методология, инструментарий и интеграция ML
 - 2.1 Схема эксперимента
 - 2.2 Обзор технологий и инструментов
 - 2.2.1 Архитектура микросервисов: определение и обоснование
 - 2.2.2 Докерная обработка и контейнеризация
 - 2.2.3 Colima: Docker на macOS без Docker Desktop
 - 2.2.4 Kotlin и Ktor Фреймворк
 - 2.2.5 Python и FastAPI
 - 2.2.6 Аутентификация JWT: структура, валидация и совместное использование между службами
 - 2.2.7 HikariCP: объединение соединений с базой данных
 - 2.2.8 Мультиплатформенность Kotlin (KMP)
 - 2.2.9 SQLDelight: типобезопасный SQL для KMP
 - 2.2.10 Streamlit: быстрая разработка информационной панели
 - 2.2.11 Коин: внедрение зависимостей для Kotlin
 - 2.2.12 Ollama: локальный вывод LLM
 - 2.2.13 Ngrok: безопасное туннелирование для местных служб
 - 2.2.14 Визуализация: платформа облачного развертывания
 - 2.2.15 Neon PostgreSQL: Бессерверная управляемая база данных
 - 2.2.16 Сопрограммы Kotlin: Структурированный параллелизм
 - 2.2.17 Gradle и Shadow JAR
 - 2.2.18 Сводная таблица

- 2.3 Стратегия обработки данных: данные, генерируемые приложением
- 2.4 Конвейер анализа на основе LLM
- 2.5 Оперативные виды проектирования и анализа
- 2.6 Показатели оценки
- 2.7 Анализ результатов
- 2.8 Анализ ошибок и извлеченные уроки
- 2.9 Краткое изложение концепции
- Глава 3: Архитектура и реализация
 - 3.1 Системные проектные решения и архитектурные принципы
 - 3.2 Архитектура микросервисов
 - 3.3 Аутентификация и межведомственная идентификация
 - 3.4 Модель данных и стратегия хранения
 - 3.5 Реализация сервиса
 - Мобильное приложение объемом 3,6 КМ
 - 3.6.1 Трехслойная чистая архитектура
 - 3.6.2 Цели и конфигурация платформы
 - 3.6.3 Локальная база данных: SQLDelight схема
 - 3.6.4 Аутентификация
 - 3.6.5 Проекты
 - 3.6.6 Экран самостоятельного управления
 - 3.6.7 Задачи
 - 3.6.8 Привычки
 - 3.6.9 Процедуры
 - 3.6.10 Таймер для приготовления Помидоров
 - 3.6.11 Журнал
 - 3.6.12 Аналитические данные
 - 3.6.13 Навигация и маршрутизация
 - 3.6.14 Адаптивный макет
 - 3.6.15 Внедрение зависимостей
 - 3.6.16 Протокол автономной синхронизации
 - 3.6.17 Текущее состояние реализации
 - 3.7 Определение области применения MVP
 - 3.8 Планируемые сервисы и функции
 - 3.8.1 Расширенный самоменеджмент: помимо задач, привычек и рутины
 - 3.8.2 Расширенный таймер: общий фокус и полосы
 - 3.8.3 Социальный слой (psocial_social)

- 3.8.4 Служба профилей пользователей (psocial_user)
 - 3.8.5 Проприетарные модели ML: стратегия накопления данных
- 3.9 Веб-панели мониторинга
- 3.10 Развертывание
- 3.11 Тестирование
- 3.12 Интеграция сервисов: сквозные потоки запросов
- 3.13 Развертывание: подробное описание Docker, Colima и рендеринга
- 3.14 Соображения безопасности
- 3.15 Окончательная проверка
- Заключение
- Список литературы

Введение

Актуальность проблемы

Мы живем в эпоху беспрецедентного доступа к инструментам самосовершенствования. Современный человек, стремящийся повысить свою личную эффективность, имеет доступ к десяткам специализированных приложений для удовлетворения любых мыслимых потребностей в производительности: менеджеры задач для отслеживания рабочих задач, трекеры привычек для построения поведенческих рутин, таймеры Pomodoro для целенаправленных рабочих сессий, приложения для ведения журнала для размышлений и помощники на базе искусственного интеллекта для получения советов. Однако такое распространение инструментов создало парадокс. Само изобилие опций стало собственным препятствием: вместо того, чтобы обеспечивать ясность и сосредоточенность, фрагментированная экосистема программного обеспечения для повышения производительности вынуждает пользователей управлять своими инструментами для повышения производительности в качестве второй работы, переключаясь между приложениями, вручную выверяя данные и создавая целостную картину собственной производительности из разрозненных источников.

Основная проблема заключается в фрагментации данных. Пользователь, который выполняет задачу в своем диспетчере задач, тратит сорок минут на сеанс Pomodoro, связанный с этой задачей, и отражает день в своем журнале, генерирует три отдельных события данных в трех отдельных приложениях, которые ничего не знают друг о друге. Диспетчер задач видит завершенный элемент. Таймер видит сеанс определенной продолжительности. Журнал видит только то, что пользователь вводит вручную. Ни одна из этих систем не может сказать пользователю: “Вы потратили сорок минут на выполнение этой высокоприоритетной задачи, вы ее выполнили, и исторически ваши самые продуктивные занятия приходятся на утро вторника - вот что это означает для того, как вам следует планировать завтрашний день”.

Такая фрагментация сопряжена с конкретными издержками. Исследования и отзывы пользователей неизменно указывают на то, что переключение контекста между приложениями является дорогостоящим с когнитивной точки зрения. Пользователи сообщают, что тратят значительное время на избыточный ввод данных в разных системах, а умственные затраты на обслуживание нескольких приложений часто приводят к отказу от одного или нескольких инструментов. Что еще более важно, разрозненный характер этих систем означает, что наиболее ценная информация — междоменные корреляции между моделями концентрации внимания, постоянством привычек, частотой выполнения задач и

общей производительностью — остается невидимой. Каждое приложение может рассуждать только о своем собственном фрагменте данных.

Таким образом, возможность заключается не просто в создании другого приложения для повышения производительности, но и в создании инфраструктуры, устраняющей фрагментацию в корне: единой платформы, где каждое действие по повышению производительности обогащает единый контекст данных и где уровень анализа искусственного интеллекта имеет одновременный доступ ко всему этому контексту при генерации аналитических данных. Это проблема, которую призвана решить компания ProductiveSocial.\

Значимость и миссия проекта

Значение ProductiveSocial выходит за рамки непосредственного удобства замены нескольких приложений одним. Проект представляет собой фундаментальный сдвиг во взаимоотношениях между пользователем и его инструментами повышения производительности — от пассивного отслеживания к активному анализу с учетом контекста.

Современные приложения для повышения производительности, даже самые сложные, остаются по сути пассивными хранителями записей. Они сохраняют то, что вводит пользователь, и отображают это с разной степенью наглядности. Даже функции ИИ в существующих инструментах ограничены собственными рамками данных приложения: диспетчер задач с помощью ИИ может только рассуждать о задачах. Отслеживание привычек с аналитикой может выявлять только закономерности в данных о привычках. Ни один из них не может переступить черту, чтобы спросить: “Как менялись показатели выполнения задач в те дни, когда эта привычка была выработана? И в те же дни, как долго длились сеансы Pomodoro и сколько из них было заброшено?”

ProductiveSocial стремится стать первой платформой, которая сделает это междоменное рассуждение способом обратной связи с пользователями по умолчанию. Когда пользователь запускает сводный анализ производительности, искусственный интеллект получает полный контекст производительности пользователя: каждую задачу и ее статус, каждую привычку и частоту ее выполнения, каждую рутину и ее соблюдение, а также полную историю сеансов фокусирования, включая частоту завершения, продолжительность работы, часы пик и модели отказа. Такая широта контекста позволяет получить качественно инсайдерскую информацию, которая невозможна ни в одном изолированном инструменте, независимо от того, насколько сложной может быть его собственная внутренняя аналитика.

Миссия выражена фразой, которая с самого начала определяла дизайн проекта: превратить инструменты повышения производительности из пассивных

контейнеров данных в интеллектуальную операционную систему для себя. Это обрамление имеет прямое архитектурное значение. Операционная система не просто хранит программы; она управляет их ресурсами, координирует их взаимодействие и предоставляет пользователю единый интерфейс для всех возможностей компьютера.

Социальное измерение платформы - модуль для обмена достижениями в области производительности, схемами привычек и схемами фокусировки с сообществом пользователей — добавляет дополнительный уровень значимости. Социальная ответственность - один из наиболее эмпирически подтвержденных механизмов сохранения привычек и достижения целей. Проектируя социальный слой, архитектурно отделенный от рабочего пространства для повышения производительности (предотвращая отвлекающее и навязчивое поведение, характерное для основных социальных сетей), ProductiveSocial стремится использовать мотивационную силу сообщества, не внося фрагментации внимания, которая в первую очередь подрывает производительность.

Цели и задачи

Основной целью этого проекта является разработка и внедрение отказоустойчивой платформы микросервисов, которая объединяет основные инструменты управления личной производительностью в целостную экосистему, улучшенную искусственным интеллектом.

Эта цель подразделяется на следующие конкретные задачи:

Задача 1 — Разработать архитектуру микросервисов. Определите границы служб, протоколы связи, модель аутентификации и правила владения данными для системы независимых, но совместимых внутренних служб. Каждая служба должна быть независимо разворачиваемой, независимо масштабируемой и способной продолжать работу даже тогда, когда другие службы временно недоступны.

Задача 2 — Внедрение основных служб повышения производительности. Создайте и разверните службу самостоятельного управления, отвечающую за управление производительностью пользователей. Создайте и разверните службу таймера, службу отслеживания сеансов Pomodoro focus.

Задача 3 — внедрить конвейер аналитики искусственного интеллекта. Создайте службу, которая организует сбор данных из всех других служб, создает структурированные запросы, вызывает вывод LLM через службу выставления счетов и сохраняет аналитические отчеты.

Задача 4 — Внедрение инфраструктуры кредитования и логического вывода. Создание внутренней бухгалтерской книги и механизма логического вывода. Поддерживайте трех поставщиков LLM (Ollama, Anthropic Claude, OpenAI GPT) и модель pathwaysklearn для будущего использования. Создайте кредитную систему в виде книги учета транзакций с автоматической инициализацией приветственного кредита.

Задача 5 — Создание административного и пользовательского интерфейсов. Первоначально внедрите панель администратора на основе Streamlit для управления системой и панель пользователя для доступа к данным о производительности, аналитике фокуса, анализу искусственного интеллекта и кредитному балансу.

Задача 6 — Реализовать синхронизацию сначала в автономном режиме. Разработайте и внедрите идемпотентный протокол пакетной синхронизации как для службы самостоятельного управления, так и для службы таймера, позволяющий мобильным клиентам работать без подключения к сети и надежно синхронизировать данные при повторном подключении.

Задача 7 — сквозная проверка системы. Проведите эмпирическое тестирование всего конвейера анализа от ввода данных до вывода LLM, оцените качество сгенерированных аналитических данных по сравнению с базовым уровнем, основанным на правилах, и проверьте правильность всех взаимодействий служб, включая списание кредитов, межведомственную аутентификацию JWT и целостность автономной синхронизации.

Новизна

Новизна продуктивного общения заключается в четырех взаимосвязанных измерениях.

Архитектурная новизна — единый контекст производительности для рассуждений об ИИ. Существующие приложения с улучшенной производительностью, основанные на ИИ, ограничивают свой ИИ данными, принадлежащими одной службе. Диспетчер задач с ИИ может рассуждать только о задачах. Трекер привычек с ИИ может рассуждать только о привычках. ProductiveSocial разработан специально для того, чтобы предоставить уровню анализа искусственного интеллекта одновременный доступ ко всем измерениям контекста продуктивности пользователя: задачам, привычкам, распорядку дня и данным сеанса фокусировки, агрегированным на момент анализа. Этот междоменный контекст является не функцией пользовательского интерфейса, а архитектурной гарантией, обеспечиваемой внутренним дизайном API службы аналитики, которая явно выполняет выборку из всех вышестоящих служб перед любым вызовом LLM.

Методологическая новизна — обучающие данные, генерируемые приложением, как первоклассная цель проектирования. В отличие от существующих инструментов повышения производительности, которые рассматривают искусственный интеллект как неотъемлемую функцию, ProductiveSocial рассматривает накопление данных о поведении пользователей как основную цель проектирования. Каждое завершение задачи, журнал привычек, рутинный шаг и интервал сеанса фокусировки хранятся в структурированной схеме с отметками времени и перекрестными ссылками. Это не случайно; это преднамеренная основа для будущего перехода от анализа, основанного на LLM, к проприетарным прогностическим моделям, обученным на реальных моделях поведения пользователей. Путь вывода в архитектуре sklearn в службе выставления счетов существует специально для обеспечения этого перехода без каких-либо изменений в более широкой системе.

Подход к валидации

Проверка организована на трех уровнях:

Уровень 1 — Функциональная корректность. Для каждого поведения базовой системы (аутентификация JWT, учет кредитоспособности, идемпотентность синхронизации, межведомственная коммуникация) мы определяем критерий прохождения / неудачи и тестируем на соответствие ему с помощью документированных входных и выходных данных. Эти тесты подтверждают, что система выполняет то, для чего она предназначена.

Уровень 2 — Качество интеграции. Для конвейера сквозного анализа - наиболее сложного и наиболее ценного поведения в системе - мы определяем базовый уровень (сводка на основе правил), условие сравнения (анализ, сгенерированный LLM), метрику (оценка релевантности человеком по шкале 1-5) и документированный результат с примерами входных и выходных данных.

Уровень 3 — Эксплуатационная надежность. Для развертывания на бесплатном уровне Render мы измеряем задержку холодного запуска, время отклика в режиме ожидания и поведение в условиях недоступности службы (например, когда служба таймера недоступна во время вызова аналитики).

Схема эксперимента и результаты полностью задокументированы в главе 2.

Критерии успеха

Критерии успеха для этого проекта определены по трем измерениям, каждое из которых имеет конкретный метод измерения:

1. Корректность системной интеграции.

- Все серверные службы успешно взаимодействуют, используя определенные протоколы (JWT для связи клиент-сервис, X-Internal-Key для связи сервис-сервис).
- Вся информация о производительности пользователя успешно агрегируется и передается в LLM в виде единого запроса аналитики.
- Списание кредита является математически корректным и регистрируется в журнале транзакций.
- Измерение: функциональный тест с документированными парами ввода / вывода. Цель: 100%-ная скорость прохождения.

2. Качество анализа искусственного интеллекта.

- Анализы, сгенерированные LLM, оцениваются на основе базовых показателей, основанных на правилах, на предмет релевантности, специфичности и возможности принятия мер.
- Результат получения степени магистра права должен быть выше базового уровня по шкале оценки персонала (1-5) с разницей не менее чем в 1,5 балла.
- Измерение: параллельное сравнение с документально подтвержденной оценкой. Цель: балл LLM $\geq 3,5 / 5$ по сравнению с базовый уровень $\leq 2,0 / 5$.

3. Надежность синхронизации.

- Автономная синхронизация корректно сохраняется и согласовывает все типы данных после имитированного цикла отключения-повторного подключения.
- Гарантия идемпотентности: повторная отправка одного и того же запроса на синхронизацию не приводит к дублированию записей.
- Измерение: тест с заранее определенным набором данных из 32 сеансов, 55 отработанных навыков, 8 заданий. Цель: корректная синхронизация 100% записей с нулевым количеством дубликатов при повторной попытке.

Глава 1: Анализ предметной области и видение решения

1.1 Анализ предметной области

Область управления личной продуктивностью находится на пересечении поведенческой психологии, взаимодействия человека и компьютера и прикладного искусственного интеллекта. Понимание предметной области требует четкого представления о типах продуктивной работы, которой занимаются пользователи, о когнитивных и мотивационных механизмах, управляющих поведением в этой области, и о технологическом ландшафте, который возник для ее поддержки.

Категории производительности работают. На самом широком уровне инструменты повышения производительности учитывают четыре различные категории поведения пользователей:

Целенаправленное управление задачами предполагает разложение целей на отдельные элементы, требующие выполнения, с указанием сроков, приоритетов и состояний завершения. Это старейшая и наиболее развитая категория инструментов для повышения производительности, уходящая корнями в методологию Getting Things Done (GTD) и более ранние бумажные системы. Пользователям в этом режиме необходимо быстро захватывать рабочие элементы, упорядочивать их по контексту и приоритету, отслеживать завершение и отображать наиболее важные элементы в любой данный момент.

Формирование привычек и распорядка дня включает в себя установление и поддержание повторяющегося поведения — как разовых действий в день (чистка зубов, медитация), так и структурированных последовательностей шагов, выполняемых по порядку (утренняя рутина, протокол учебного занятия). В отличие от управления задачами, которое ориентировано на завершение и закрытость, отслеживание привычек ориентировано на последовательность и поддержание последовательности действий. Механика мотивации иная: награда - это не завершение, а непрерывность, а риск - это не отставание, а прерывистость. Процедуры добавляют временное и последовательное измерение: не просто “сделал ли я это”, но “выполнил ли я все шаги в правильном порядке и за нужное время”.

Сеансы целенаправленной работы представляют собой особый способ повышения производительности, при котором пользователь обязуется непрерывно концентрироваться на одной задаче в течение определенного интервала. Методика Помодоро, разработанная Франческо Чирилло в 1980-х годах, оформила это в широко принятый протокол: работайте 25 минут, сделайте 5-минутный перерыв, повторите. Появляется множество вариантов с разной длительностью, и отдельные пользователи обычно выбирают длину

интервала, соответствующую их личной концентрации внимания. Ключевым показателем в этой области является не завершение (задачи) или последовательность (привычки), а глубина вовлеченности — общее время сосредоточения, скорость завершения сеанса и отсутствие перерывов.

рефлексивное ведение дневника выполняет принципиально иную психологическую функцию, чем вышеупомянутые три категории. Вместо того чтобы продвигать действия вперед, ведение дневника создает пространство для ретроспективного осмысления: объединения событий дня в связное повествование, обработки эмоций и извлечения уроков. В контексте продуктивности ведение дневника наиболее ценно, когда оно связывает субъективный опыт прошедшего дня (что он чувствовал) с объективным отчетом о деятельности (что было фактически достигнуто). Эта связь - именно то, чего ручное ведение журнала не может сделать без посторонней помощи и что делает возможным синтез с помощью искусственного интеллекта.

Поведенческая экономика инструментов повышения производительности. Важным, но часто упускаемым из виду аспектом этой области является разрыв между внедрением инструментов и их последовательным использованием. Исследования моделей использования приложений для повышения производительности постоянно показывают, что большинство пользователей, которые загружают новое приложение для повышения производительности, отказываются от него в течение первых двух недель. Приводимыми основными причинами являются: накладные расходы на ввод данных, отсутствие видимой ценности от введенных данных и трудности при переключении контекста между несколькими приложениями. Эти шаблоны имеют прямое значение для проектирования системы: эффективная платформа для повышения производительности должна сводить к минимуму трудности при вводе данных, максимизировать видимую отдачу от введенных данных и сокращать количество приложений, которыми должен управлять пользователь.

Теория самоопределения и внутренняя мотивация. Исследования в области поведенческой психологии помогают понять, почему инструменты повышения производительности работают успешно или терпят неудачу. Теория самоопределения (Deci and Ryan, 1985) выделяет три фундаментальные психологические потребности, которые стимулируют устойчивое вовлечение в любую деятельность: автономия (чувство выбора и сопричастности), компетентность (восприятие роста и мастерства) и связанность (связь с другими людьми, разделяющими схожие цели). Приложения для повышения производительности, которые удовлетворяют этим потребностям — предоставляя пользователям контроль над своей системой, делая заметным прогресс значимыми способами и подключая их к сообществу ответственных сотрудников, — обеспечивают значительно более длительное удержание, чем те,

которые удовлетворяют только инструментальную потребность “довести дело до конца”.

Методика Помодоро успешна отчасти потому, что она напрямую затрагивает аспект компетентности: каждое завершённое занятие является видимой единицей достигнутого фокуса, а накопление завершённых циклов с течением времени создает конкретное представление о дисциплинированной работе. Специалисты по отслеживанию привычек добиваются успеха, когда визуализируют полосы, потому что эта полоса создает динамику неприятия потерь — прерывание 14-дневной полосы кажется дорогостоящим, что мотивирует к продолжению. Ни один из методов не работает надежно изолированно; их эффект усиливается, когда пользователь может видеть, что вырабатываемые им привычки коррелируют с целенаправленными рабочими сессиями, которые приводят к выполнению наиболее важных задач.

Когнитивная нагрузка и последствия для дизайна. Теория когнитивной нагрузки (Sweller, 1988) проводит различие между внутренней когнитивной нагрузкой (присущей сложности выполняемой задачи), посторонней когнитивной нагрузкой (накладными расходами, обусловленными плохим дизайном инструментов) и естественной когнитивной нагрузкой (умственными усилиями, которые приводят к обучению и долговременной памяти). Фрагментированный стек производительности создает значительную дополнительную когнитивную нагрузку: пользователь должен помнить об обновлении нескольких систем, должен мысленно интегрировать информацию из разных источников и должен управлять когнитивным переходом между различными контекстами приложения. Интегрированная платформа сводит эту внешнюю нагрузку к практически нулю — пользователь взаимодействует с одной системой, а система обрабатывает интеграцию данных внутри компании.

Роль искусственного интеллекта в производительности. Интеграция искусственного интеллекта в инструменты повышения производительности значительно ускорилась с момента массового появления больших языковых моделей в 2022-2023 годах. Текущие приложения искусственного интеллекта в этой области делятся на несколько категорий: анализ входных данных на естественном языке (преобразование устных или печатных описаний в структурированные записи задач), оптимизация планирования (рекомендации, когда планировать задачи, исходя из календаря и рабочей нагрузки), распознавание образов (определение того, какие привычки связаны с днями с высокой производительностью) и генеративное обобщение (создание письменных сводок журналов активности). Общим для всех текущих реализаций ограничением является проблема границ данных, описанная во введении: каждая функция искусственного интеллекта может рассуждать только о данных, принадлежащих приложению, в котором она размещена.

Целевые профили пользователей. Чтобы обосновать проектные решения с учетом конкретных потребностей пользователей, в ходе анализа предметной области были определены три репрезентативных профиля пользователей:

Профиль А — Работник умственного труда. Профессионал с высокими когнитивными нагрузками (разработчик программного обеспечения, академический исследователь, писатель), которому необходимо управлять постоянным накоплением сложных задач, поддерживать набор ежедневных профессиональных привычек (сосредоточенное чтение, ежедневное письмо, физические упражнения) и отслеживать, как распределяется его время сосредоточения по проектам. Основной болевой точкой этого пользователя является неспособность ответить на вопрос: “Трачу ли я свое время на то, чтобы сосредоточиться на правильных вещах?” Они технически сложны и будут использовать API, если пользовательского интерфейса недостаточно.

Профиль В — Студент. Студент университета, контролирующий сроки выполнения курсовых работ, учебные привычки (ежедневный пересмотр, конспекты лекций, практические задачи) и процедуры подготовки к экзаменам. Основная задача этого пользователя - сбалансировать множество параллельных обязательств с непредсказуемыми рабочими нагрузками. Они высоко мотивированы сериями и элементами геймификации, но быстро отказываются от приложений, если затраты на ввод данных высоки. Они используют свой телефон в качестве основного вычислительного устройства.

Профиль С — Энтузиаст самосовершенствования. Пользователь, в первую очередь мотивированный формированием привычек и оптимизацией образа жизни, а не профессиональным управлением задачами. Их основным использованием платформы будет отслеживание привычек (физические упражнения, медитация, питание, чтение) с периодическим анализом ИИ закономерностей согласованности и корреляции между привычками и общим самочувствием.

Эти три профиля послужили основой для определения приоритетов функций в разделе 1.5 и принятия оперативных инженерных решений в разделе 2.5.

1.2 Анализ проблемы "КАК ЕСТЬ"

Текущее состояние рынка инструментов для повышения личной производительности можно охарактеризовать тремя структурными проблемами, которые адекватно не решены ни одним существующим решением.

Проблема 1: Изолированные данные и фрагментированный пользовательский контекст

Фундаментальным архитектурным ограничением текущего рынка является то, что каждое производительное приложение поддерживает свое собственное изолированное хранилище данных. Пользователь, использующий Todoist для управления задачами, Habitica для отслеживания привычек, Forest для определения времени фокусировки и Day One для ведения журнала, имеет четыре отдельных приложения, четыре отдельных учетные записи пользователей и четыре отдельных базы данных. Не существует механизма, с помощью которого какая-либо из этих систем могла бы наблюдать за событиями, происходящими в других.

Это создает дополнительную проблему для анализа ИИ. Даже в самых сложных текущих реализациях помощник ИИ в диспетчере задач может только наблюдать за задачами. Он не может знать, что пользователь потратил 40 минут сосредоточенности на конкретной задаче этим утром, что привычка к ежедневным упражнениям, которая должна была сопровождать это рабочее занятие, была пропущена сегодня, или что пользователь постоянно отказывался от занятий сосредоточенностью после второго Pomodoro в течение последних двух недель. Каждый из этих фактов фиксируется в какой-либо системе, но ни одна система не имеет доступа ко всем из них одновременно.

В результате пользователи получают фрагментарную обратную связь без учета контекста. Диспетчер задач сообщает вам, что вы делали; отслеживание привычек сообщает вам, были ли вы последовательны; таймер показывает, как долго вы сосредотачивались. Ни один из них не может рассказать вам, как взаимодействуют эти измерения или что междоменные паттерны в вашем поведении подсказывают о том, как вам следует скорректировать свой подход.

Проблема 2: Накладные расходы на ручной ввод данных и переключение контекста

Пользователи, пытающиеся поддерживать несколько приложений для повышения производительности, сталкиваются со значительной административной нагрузкой. Одна и та же информация часто должна вводиться в нескольких местах: задача, выполненная в диспетчере задач, также должна быть отражена в журнале, система отслеживания привычек должна отмечать рабочий сеанс, который сопровождал ее, а журнал таймера должен содержать ссылку на задачу, которая была в центре ее внимания. На практике пользователи либо дублируют ввод данных (увеличивая накладные расходы), либо соглашаются с несогласованностью между системами (уменьшая ценность каждой из них), либо отказываются от всех приложений, кроме одного (теряя охват важных измерений).

Затраты на переключение контекста при переходе от приложения к приложению - это не просто временные затраты; это когнитивные затраты.

Исследование остаточного внимания (феномена, при котором внимание задерживается на предыдущем приложении даже после переключения) предполагает, что каждое переключение контекста наносит ощутимый урон когнитивным ресурсам. Для пользователя, который уже стремится сохранить сосредоточенность, требование управлять четырьмя отдельными производительными приложениями само по себе является препятствием для состояния, которое эти приложения должны поддерживать.

Проблема 3: Отсутствие Междисциплинарного Поведенческого интеллекта

Текущий искусственный интеллект производительности локально оптимален, но глобально слеп. Каждое приложение может предоставлять сложную аналитическую информацию в своей собственной области — Todoist может рекомендовать расстановку приоритетов задач на основе крайних сроков и исторических показателей выполнения; Habitica может предсказать, какие привычки подвержены риску нарушения, на основе данных о последовательности действий; Forest может показывать шаблоны фокусировки по времени суток. Но ни одно приложение не может спросить: “В те дни, когда ваша утренняя зарядка была завершена, как менялась скорость выполнения ваших дневных задач? И каков был уровень отказа от помидоров в те дни по сравнению с днями, когда вы отказывались от физических упражнений?”

Такого рода междоменные запросы - это именно тот тип инсайта, который имеет наибольшую практическую ценность для изменения поведения. Поведенческие исследования последовательно показывают, что привычки не действуют изолированно — они образуют взаимозависимые сети, в которых завершение или отказ от одной привычки оказывает вероятностное влияние на другие в тот же день. Без системы, учитывающей полный поведенческий контекст, эти взаимозависимости остаются невидимыми, и помощь, получаемая пользователями с помощью искусственного интеллекта, неизбежно будет более поверхностной, чем это технически возможно.

1.3 Обзор существующих решений

Чтобы определить конкурентную среду и продемонстрировать конкретный пробел, который заполняет ProductiveSocial, мы проанализировали семь ведущих приложений, представляющих различные категории инструментов для повышения производительности. В анализе рассматриваются шесть аспектов: возможность управления задачами, возможность отслеживания привычек, встроенная функциональность focus / Pomodoro, функции анализа ИИ, автономная архитектура и имеет ли ИИ доступ к унифицированным межмодульным данным.

Todoist - лидер рынка управления задачами, насчитывающий более 30 миллионов пользователей. Он предоставляет сложную систему задач, ориентированную на GTD, с проектами, ярлыками, уровнями приоритета, повторяющимися задачами и анализом даты на естественном языке. Его функции искусственного интеллекта (“AI Assistant”) сосредоточены на переформулировании задач, предложениях по декомпозиции и автоматическом планировании. Однако в Todoist нет встроенного отслеживания привычек, нет функциональности Pomodoro, а его искусственный интеллект полностью ограничен областью задач. Пользователи, которые хотят отслеживать привычки вместе с Todoist, должны интегрировать отдельное приложение, и межприложенческий анализ с помощью искусственного интеллекта недоступен.

Habitica геймифицирует выработку привычек, превращая привычки, ежедневные задачи и поручения в ролевую игру. У него большое и заинтересованное сообщество и социальные функции, которые действительно используются для обеспечения подотчетности. Однако его управление задачами является рудиментарным по сравнению со специализированными менеджерами задач, у него нет функциональности Pomodoro и у него нет каких-либо функций анализа искусственного интеллекта. Его социальные функции специально разработаны на уровне геймификации и не могут быть перенесены в негеймифицированный контекст.

Forest - это специализированное приложение Pomodoro timer с метафорой посадки: пользователи выращивают виртуальные деревья во время сеансов фокусировки, которые погибают, если они покидают приложение. Он очень эффективен в качестве устройства для привлечения внимания, но не поддерживает управление задачами или отслеживание привычек, а его аналитика ограничена графиками истории сеансов. Анализ с помощью искусственного интеллекта отсутствует, и данные недоступны вне приложения.

Concept - это самый близкий из существующих продуктов к концепции универсальной платформы для повышения производительности. Его гибкость — пользователи могут создавать собственные базы данных, связывать записи между страницами и создавать любую организационную структуру, какую только могут себе представить, — сделала его действительно полезным в качестве единого рабочего пространства для многих пользователей. Однако эта гибкость также является ее ограничением в области производительности: в Notion отсутствует встроенная механика отслеживания привычек, для управления задачами требуется ручная настройка, а не предоставление специализированных структур производительности, для функциональности Pomodoro требуются встраивания сторонних производителей, а функции искусственного интеллекта (Notion AI) в первую очередь ориентированы на редактирование текста и обобщение, а не на анализ поведенческих паттернов.

Важно отметить, что ИИ Concept может рассуждать только о содержании страниц Concept, а не о междоменных поведенческих паттернах, полученных из структурированных журналов активности.

Day One - ведущее приложение для ведения журнала для iOS и macOS с мощными функциями конфиденциальности, поддержкой медиа и пометкой местоположения. Он отлично подходит для ведения рефлексивного журнала вручную, но в нем нет управления задачами, отслеживания привычек, функций Pomodoro и искусственного интеллекта, который мог бы синтезировать данные об активности из других источников. Его возможности искусственного интеллекта ограничены транскрипцией и базовой поддержкой текста.

RescueTime - это приложение для автоматического отслеживания рабочего времени, которое пассивно отслеживает, на какие приложения и веб-сайты тратит время пользователь, и генерирует показатели производительности и отчеты. Это решает другую проблему — пассивный мониторинг фактического использования компьютера, а не намеренное отслеживание запланированных действий. В нем нет управления задачами, отслеживания привычек и таймера Pomodoro. Его функции искусственного интеллекта создают автоматические еженедельные отчеты на основе данных об использовании компьютера. Он не интегрируется с менеджерами задач или трекерами привычек.

Sunsama - это приложение для ежедневного планирования, предназначенное для работников умственного труда. Он интегрируется с Todoist, Asana, GitHub и другими внешними сервисами для объединения задач в единый ежедневный просмотр с поддержкой ограничения по времени. Он обладает базовыми функциями фокусировки в стиле Pomodoro и генерирует ежедневные сводки “выключения”. Однако в нем нет отслеживания привычек, поддержка Pomodoro является базовой, его искусственный интеллект ограничен суммированием задач дня, и для полноценной интеграции требуются платные подписки на внешние сервисы.

Сравнение приведено в следующей таблице:

Feature	Todoist	Habitaica	Forest	Notion	Day One	RescueTime	Sunsama	ProductiveSocial
Task Management	✅ Full	⚠️ Basic	❌	✅ Manual setup	❌	❌	✅ Full	✅ Full
Habit Tracking	❌	✅ Full	❌	⚠️ Manual setup	❌	❌	❌	✅ Full (Start/Quit, subtasks, completion logs)
Pomodoro Timer	❌	❌	✅ Full	⚠️ Plugin only	❌	❌	⚠️ Basic	✅ Full (adaptive, abandonment risk, entity-linked)
AI Analysis	⚠️ Task-only	❌	❌	⚠️ Text editing	⚠️ Basic	⚠️ Usage reports	⚠️ Day summary	✅ Full cross-domain
Offline-First	✅	❌	⚠️	⚠️ Partial	✅	❌	❌	✅ Full batch sync
Social / Community	❌	✅ Gamified	❌	❌	❌	❌	❌	📅 Planned
Unified AI Context	❌	❌	❌	❌	❌	❌	❌	✅ (tasks + habits + routines + focus)
Open LLM Provider	❌	❌	❌	❌	❌	❌	❌	✅ (Ollama/Anthropic/OpenAI)

Утверждение в этом отчете о том, что “архитектура, объединяющая все типы отдельных инструментов повышения производительности с единым уровнем анализа искусственного интеллекта”, является новой, подтверждается этим сравнением. Хотя отдельные модули не являются чем-то новым по отдельности, их комбинация в рамках единой модели данных с междоменным доступом искусственного интеллекта представляет собой системный дизайн, которого не существует на текущем рынке.

1.4 Видение решения

Решение ProductiveSocial решает три выявленные проблемы с помощью определенного набора архитектурных и продуктовых решений.

Для решения проблемы разрозненности данных: Архитектура микросервисов, в которой все сервисы используют общую идентификацию пользователя, и выделенная служба аналитики, которая объединяет данные по всем сервисам перед вызовом анализа с помощью искусственного интеллекта. Пользователь не интегрирует внешние сервисы; все модули встроены в платформу, а служба аналитики имеет гарантированный внутренний доступ ко всем данным через межведомственный уровень связи, защищенный общим секретом.

Для решения проблемы ввода данных и переключения контекста: единое приложение со специально созданными пользовательскими интерфейсами для каждого модуля производительности, подключенное к общей модели данных. Задача, созданная в диспетчере задач, автоматически становится доступной в качестве целевой ссылки в таймере, и время, зарегистрированное в таймере, автоматически записывается обратно в задачу. Автономная архитектура с пакетной синхронизацией гарантирует, что пользователь может использовать приложение без подключения к сети, устраняя общую проблему для мобильных инструментов повышения производительности.

Для решения задачи междоменного анализа: Служба аналитики, которая собирает полный контекст производительности на момент анализа и передает его в большую языковую модель с подсказками для конкретных типов, предназначенными для получения междоменной информации. ИИ рассуждает не только о задачах или сеансах фокусировки; он рассуждает о полном поведенческом профиле пользователя, который фиксируется интегрированной платформой.

1.5 Определение приоритетов

Функции продукта были расставлены по приоритетам с использованием концепции соотношения ценности и затраченных усилий с дополнительным ограничением, заключающимся в том, что система должна функционировать как единое интегрированное целое на каждом этапе - частичные реализации, нарушающие поток данных между модулями, неприемлемы.

Приоритет 1 (должен быть у MVP):

- Аутентификация и идентификация пользователя
- Управление задачами с проектами, подзадачами, приоритетами и автономной синхронизацией
- Отслеживание привычек (типы запуска / выхода, журналы завершения, подзадачи)
- Рутинное управление с упорядоченными шагами
- Жизненный цикл сеанса Pomodoro (создание, интервал запуска, завершение, отмена)
- Конвейер межведомственного анализа с использованием искусственного интеллекта, включающий не менее четырех типов анализа

- Кредитная система с регистрацией транзакций и приветственным депозитом
- Панель администратора для управления моделями и кредитами
- Панель управления пользователя для доступа к данным и запуска анализа.

Приоритет 2 (важный, но после MVP):

- Функции адаптивного таймера (риск отказа, предложения о перерыве, рекомендации по продолжительности)
- анализ структуры фокусировки (по часам суток, дням недели)
- Тип сводного анализа по недельному таймеру
- Пользовательский тип анализа с пользовательской подсказкой
- Автономная синхронизация сеансов по таймеру
- Мобильное приложение с полным охватом функций

Приоритет 3 (Будущее):

- Социальные функции (общедоступные профили, обмен достижениями, схемы привычек, лента сообщества, партнеры по подотчетности)
- Собственные обученные модели ML для прогнозирования поведения (вероятность сохранения привычки, вероятность завершения задачи, прогнозирование продолжительности сеанса фокусировки),
- Модуль ведения дневника с отслеживанием настроения и обобщением дня с помощью искусственного интеллекта
- Модуль заметок со связыванием контекстных объектов
- Служба профилей пользователей, владеющая отображаемыми именами, аватарами, биографиями и графой социальных сетей, расширенными функциями самостоятельного управления: списки дел (упрощенные одноэтапные элементы, отличные от полных задач), быстрый просмотр входящих сообщений, просмотр календаря, контрольные списки, шаблоны
- Расширенные функции таймера: общие комнаты фокусировки, полосы фокусировки, цели сеанса,
- Сторонние интеграции (Google Calendar, носимые устройства, Apple Health)

- Вывод модели на устройстве для полностью автономного анализа искусственного интеллекта

Текущий проект предоставляет все элементы приоритета 1 и большинство элементов приоритета 2. Приложение КМР является каркасным и функциональным в своем потоке аутентификации, а остальные экраны запланированы для последующей разработки. Элементы приоритета 3 отложены до будущих этапов разработки, при этом архитектура специально разработана для их размещения, не требуя структурных изменений в существующих сервисах.

Стоит подчеркнуть, что список приоритетов 3 - это не набор спекулятивных льгот для имущих - он отражает особенности, которые определяют долгосрочную дифференциацию ProductiveSocial. Социальный слой, журнал и более богатые модули для самостоятельного управления - вот что превращает платформу из мощного инструмента повышения личной продуктивности в операционную систему повышения производительности, описанную в заявлении о миссии. Четыре сервиса, созданные на этом этапе, являются фундаментом инфраструктуры, на котором будут построены все эти функции.

Глава 2: Методология, инструментарий и интеграция ML

2.1 Схема эксперимента

Эксперименты разделены на три категории: тесты функциональной корректности (проверка того, что система выполняет то, для чего она предназначена), оценка качества (сравнение анализа, сгенерированного LLM, с базовым уровнем, основанным на правилах) и эксплуатационная надежность (проверка поведения в условиях развертывания и отказа).

Все эксперименты проводились с согласованным набором тестовых данных, заданным в локальной среде разработки для пользователей *test@productivesocial.com*. Набор данных состоит из:

- 3 проекта: Рабочий, Личный, Обучающий
- 8 задач, распределенных по всем проектам, с различными приоритетами и состояниями завершения
- 6 привычек (5 ежедневных, 1 еженедельная) обоих типов: "Начать" и "Бросить"
- 4 процедуры с пошаговым отслеживанием (утро, Углубленная работа, Вечернее обучение, Еженедельный обзор)
- 32 сеанса Pomodoro (31 завершённый, 1 заброшенный), связанные с задачами, привычками, расписанием дня и автономные
- 55 завершений работы с привычками за 14 дней истории

Этот набор данных был выбран потому, что он представляет реалистично заполненный профиль пользователя — ни тривиально маленькую выборку, ни искусственно раздутую — и потому, что он охватывает все типы сущностей, на основе которых работает служба аналитики.

Полная таблица результатов эксперимента представлена в разделе 2.7.

2.2 Обзор технологий и инструментов

В этом разделе описываются все важные технологии, используемые в системе ProductiveSocial — не только сделанный выбор, но и сама концепция, — чтобы последующие архитектурные решения можно было понять в полном контексте.

2.2.1 Архитектура микросервисов: определение и обоснование

Архитектура микросервисов - это подход к разработке программного обеспечения, при котором большое приложение разбивается на набор небольших независимо развертываемых служб, каждая из которых отвечает за определенные бизнес-возможности и взаимодействует с другими службами по сети (обычно HTTP или очередь сообщений). Это контрастирует с монолитной архитектурой, в которой вся функциональность упакована в единый развертываемый модуль.

Термин был популяризирован Мартином Фаулером и Джеймсом Льюисом в статье 2014 года, в которой были определены определяющие характеристики шаблона: единая ответственность для каждой службы, независимая возможность развертывания, децентрализованное управление данными (каждая служба владеет своей собственной базой данных) и дизайн с учетом сбоев (каждая служба предполагает, что другие службы могут быть недоступны).

Почему именно микросервисы для ProductiveSocial? Решение о создании четырех отдельных сервисов, а не единого монолита, было продиктовано тремя конкретными требованиями к этой системе:

Во-первых, сервисы имеют действительно отличные технологические профили. Три сервиса Kotlin (self-manager, timer, analytics) лучше всего создавать на статически типизированном языке, основанном на JVM, поскольку их основная работа заключается в структурированном управлении данными со сложной логикой отношений. Служба выставления счетов, напротив, имеет принципиально иной профиль: ей необходимо загружать файлы моделей Pythonsklearn .pkl и вызывать библиотеки LLM SDK, созданные на Python (anthropic, openai). Для создания монолита потребовался бы выбор одной из этих технологий в ущерб другой; микросервисы позволяют каждой службе использовать наилучший язык и фреймворк для своей конкретной работы.

Во-вторых, к службам предъявляются разные требования к отказоустойчивости. Служба аналитики должна продолжать функционировать, даже если служба таймера временно недоступна — она должна просто исключить раздел Pomodoro из своего анализа, а не возвращать ошибку. Монолит не может выразить это постепенное ухудшение естественным образом; в архитектуре микросервисов служба аналитики просто улавливает тайм-аут HTTP от клиента timer и продолжает работу.

В-третьих, жизненные циклы развертывания служб действительно различаются. При внесении изменений в логику построения запросов LLM службы выставления счетов необходимо повторно использовать только службу выставления счетов. В monolith даже однострочное изменение логики

выставления счетов потребовало бы повторного развертывания всего приложения - со всеми вытекающими отсюда рисками.

Недостатки этого выбора реальны: межсервисные HTTP-вызовы приводят к задержкам в сети, распределенные системы сложнее отлаживать, чем монолитные системы, а согласованность данных за пределами служб требует тщательного проектирования. Каждый из этих компромиссов подробно рассмотрен в архитектуре, описанной в главе 3.

2.2.2 Докерная обработка и контейнеризация

Docker - это платформа для упаковки, распространения и запуска программного обеспечения в контейнерах. Контейнер - это облегченная изолированная среда выполнения, которая объединяет приложение со всем, что ему нужно для запуска: его среда выполнения (Java, Python), его зависимости (библиотеки, пакеты), его конфигурация и файловая система. В отличие от виртуальной машины, которая эмулирует всю операционную систему, контейнер использует ядро операционной системы хоста, что значительно ускоряет его запуск и снижает потребление ресурсов.

Проблема, которую решает Docker, заключается в несоответствии среды: проблема “это работает на моей машине”. Без контейнеризации служба, которая корректно работает на ноутбуке разработчика, может выйти из строя в рабочей среде из-за различий в установленной версии Python, наличия или отсутствия системной библиотеки или незначительных различий в соглашениях операционной системы о путях к файлам. Контейнер Docker устраняет этот класс проблем, делая среду выполнения частью развертываемого артефакта: если контейнер работает на ноутбуке разработчика, тот же контейнер будет работать и в рабочей среде.

Как ProductiveSocial использует Docker. Каждая из четырех внутренних служб имеет Dockerfile, который определяет ее образ контейнера. Docker Compose - это инструмент, используемый для совместного запуска всех служб в локальной среде разработки. Он считывает файл docker-compose.yml, который определяет все контейнеры (службы и базы данных), их сетевые подключения, переменные среды, сопоставления портов и проверки работоспособности. Вместо того чтобы запускать четыре службы и четыре базы данных вручную отдельными командами, одна команда docker compose up запускает всю систему.

```

# Kotlin service example (psocial_selfmanager)
FROM eclipse-temurin:17-jre-alpine
WORKDIR /app
COPY server/build/libs/server-all.jar app.jar
EXPOSE 1226
ENTRYPOINT ["java", "-jar", "app.jar"]

FROM python:3.11-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
EXPOSE 1229
CMD ["uvicorn", "main:app", "--host", "0.0.0.0", "--port", "1229"]

```

Рисунок 2.1. файл docker-compose.yml

2.2.3 Colima: Docker на macOS без Docker Desktop

Colima - это облегченная контейнерная среда выполнения для macOS, которая обеспечивает среду, совместимую с Docker, без необходимости использования Docker Desktop. Он запускает небольшую виртуальную машину Linux (с использованием платформы виртуализации Apple на Apple Silicon), в которой размещен демон Docker, и предоставляет сокет Docker, к которому могут подключаться Docker CLI и Docker Compose.

Почему Colima? Docker Desktop для Mac - это проприетарное программное обеспечение, для коммерческого использования которого в организациях выше определенного размера требуется платная лицензия, и оно требует больших ресурсов, чем Colima. Colima имеет открытый исходный код, бесплатна и обычно использует меньше системных ресурсов. Для целей разработки Colima функционально эквивалентен Docker Desktop: команды docker и docker compose работают идентично, образы контейнеров создаются и запускаются одинаково, а сопоставления портов работают так, как ожидалось.

Для запуска среды выполнения контейнера перед началом разработки требуется одна команда:

началоколиммы

Это запускает виртуальную машину Linux (примерно через 5-10 секунд) и делает команды Docker доступными в текущем сеансе терминала. Все операции docker compose для локального стека разработки ProductiveSocial в ProductiveSocial выполняются поверх этой среды, управляемой Colima.....

2.2.4 Kotlin и Ktor Фреймворк

Kotlin - это статически типизированный язык программирования, разработанный компанией JetBrains, впервые выпущенный в 2016 году. Он работает на виртуальной машине Java (JVM) и полностью совместим с Java: код Kotlin может вызывать библиотеки Java и наоборот. Kotlin стал основным языком для разработки на Android (Google объявил его предпочтительным

языком для Android в 2019 году) и получил значительное распространение при разработке на стороне сервера.

Ключевыми функциями, которые делают Kotlin правильным выбором для бэкенд-сервисов ProductiveSocial, являются:

Безопасность с нулевым значением: В Kotlin переменная не может содержать нулевое значение, если ее тип явно не объявлен как nullable (например, String? против String). Это устраняет весь класс ошибок NullPointerException, которые являются наиболее распространенным сбоем во время выполнения в приложениях Java. Для такого сервиса, как analytics, который собирает данные из нескольких вышестоящих источников, некоторые из которых могут возвращать пустые или частичные данные, гарантия компилятора, что каждое потенциально нулевое значение обрабатывается явно, является значимым свойством надежности.

Сопрограммы: Система сопрограмм Kotlin обеспечивает структурированный подход к асинхронному программированию. Сопрограмма - это вычисление, которое может быть приостановлено и возобновлено без блокирования потока, в котором оно выполняется. Это критически важно для службы аналитики, которая должна выполнять три сетевых вызова параллельно (для самостоятельного управления, таймера и, в конечном итоге, выставления счетов), не блокируя поток для каждого вызова. В традиционном коде с блокировкой потоков для трех параллельных вызовов потребовалось бы три потока. С помощью сопрограмм все три вызова могут выполняться в небольшом пуле потоков, каждый из которых приостанавливается в ожидании сети и возобновляется при поступлении ответа.

Ktor - это фреймворк для создания HTTP-серверов и клиентов в Kotlin, разработанный компанией JetBrains. В отличие от более тяжелых фреймворков, таких как Spring (которые предоставляют комплексные решения для всех возможных требований предприятия), Ktor намеренно минимален: он предоставляет DSL маршрутизации, слоты плагинов для общих задач (аутентификация, ведение журнала, сериализация, CORS) и HTTP-клиент для выполнения исходящих запросов. Он добавляет только то, что установлено.

2.2.5 Python и FastAPI

Python является доминирующим языком в экосистеме машинного обучения и науки о данных. Служба выставления счетов написана на Python специально потому, что ей необходимо интегрироваться со стекком Python ML: scikit-learn для загрузки обученных .pkl файлов моделей, joblib для десериализации модели, anthropic для вызова Anthropic Claude API и openai для вызова OpenAI GPT API.

Сначала эти библиотеки существуют в Python; их эквиваленты в других языках являются второстепенными и часто неполными.

FastAPI - это современный веб-фреймворк на Python, предназначенный для создания API. Он построен поверх Starlette (для асинхронной обработки HTTP) и Pydantic (для проверки данных). Его определяющей характеристикой является автоматическая документация API: объявляя типы запросов и ответов как Pydantic модели, FastAPI автоматически генерирует интерактивный пользовательский интерфейс Swagger и спецификацию OpenAPI. Эта документация немедленно доступна по адресу /docs в запущенной службе.

FastAPI использует синтаксис `async / await` в Python для неблокирующей обработки запросов. Для службы выставления счетов это имеет значение, когда вызов логического вывода LLM занимает 10-30 секунд: при асинхронной обработке сервер может продолжить обработку других запросов, ожидая ответа поставщика LLM, вместо того, чтобы удерживать поток в режиме ожидания.

Pydantic - это библиотека проверки данных, которая используется во всей службе выставления счетов для определения форм запросов и ответов. Когда запрос поступает в `POST /api/v1/internal/predict`, Pydantic проверяет, что `selfmanager_user_id` является целым числом, `model_id` является допустимым UUID, а `input_data` является словарем — перед запуском любой бизнес-логики. Недействительные запросы немедленно отклоняются со структурированным ответом об ошибке.

SQLAlchemy (асинхронный, через `asyncpg`) - это ORM (объектно-реляционный картограф), используемый в службе выставления счетов. Он сопоставляет классы Python с таблицами базы данных и автоматически генерирует SQL, позволяя службе запрашивать и изменять базу данных, используя объекты Python, а не необработанные строки SQL.

2.2.6 Аутентификация JWT: структура, валидация и совместное использование между службами

JWT расшифровывается как веб-токен JSON. Это компактный, автономный формат для представления утверждений (утверждений о предмете, таких как личность пользователя) способом, который может быть криптографически проверен. JWT выдается сервером аутентификации и представляется клиентом при каждом последующем запросе; принимающий сервер может проверить подлинность токена, не связываясь с сервером выдачи.

Структура токена. JWT состоит из трех секций в кодировке `base64url`, разделенных точками:

1. eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9
2. eyJ1c2VySWQiOiJyLCJlbWFpbCI6InVzZXJAZXhbbXBsZS5jb20iLCJpYXQiOiJE3MTYwMDAwMDAsImV4cCI6MTcxNjA4NjQwMH0
3. SflKxwRJSMeKKF2QT4fwpMeJf36POk6yJV_adQssw5c

Первый раздел - это **заголовок**: объект JSON, определяющий алгоритм, используемый для подписи (например, HS256, который является HMAC-SHA256) и тип токена ("JWT").

Второй раздел - это **полезная нагрузка**: объект JSON, содержащий утверждения. В ProductiveSocial полезная нагрузка содержит:

```
{
  "userId": 42,
  "email": "user@example.com",
  "iat": 1716000000,
  "exp": 1716086400
}
```

iat “выдается в” (временная метка Unix), *exp* - это время истечения срока действия. Утверждения могут быть прочитаны кем угодно — полезная нагрузка закодирована в base64, а не зашифрована. Она никогда не должна содержать конфиденциальных данных, таких как пароли.

Третий раздел - это подпись: вычисляется с помощью HMAC-SHA256 (заголовок + "." + полезная нагрузка, секрет). Секрет - это закрытая строка, известная только серверам. Подпись не может быть подделана без секрета; следовательно, действительная подпись доказывает, что токен был создан сервером, который знает секрет, и что полезная нагрузка не изменялась с момента ее выдачи.

Проверка. Когда служба получает запрос с JWT в заголовке *Authorization: Bearer <токен>*, она выполняет:

1. Расшифруйте заголовок и убедитесь, что алгоритм соответствует ожиданиям
2. Расшифруйте полезную нагрузку

3. Пересчитайте *HMAC-SHA256* (заголовок + "." + полезная нагрузка, секрет) и сравните с предоставленной сигнатурой — если они отличаются, отклоните
4. Убедитесь, что *exp* действует в будущем — если срок действия истек, отклоните
5. Извлеките *идентификатор пользователя* и *адрес электронной почты* из проверенной полезной нагрузки для использования в обработчике запроса

Совместное использование между службами. Ключевым моментом в дизайне аутентификации *ProductiveSocial* является то, что проверка JWT не требует обращения к эмитенту. Любой сервер, который знает секрет, может независимо проверить любой токен, подписанный этим секретом. Предоставляя всем трем пользовательским сервисам (*selfmanager*, *timer*, *analytics*) один и тот же *JWT_SECRET*, токен, выпущенный конечной точкой *selfmanager /api/ v1 /auth /identify*, автоматически становится действительным в службах *timer* и *analytics* - без обмена токенами, без перенаправления, без дополнительного шага входа в систему.

Этот подход с общим секретом подходит для системы, в которой все службы управляются одной организацией. Это было бы неуместно для федеративной системы, где службами управляют разные организации, и в этом случае потребовался бы более формальный поставщик удостоверений (*OAuth2*, *OpenID Connect*).

Токен доступа и токен обновления. Поток аутентификации выдает два токена: токен доступа с коротким сроком действия (используется для вызовов API) и токен обновления с длительным сроком действия (используется для получения нового токена доступа по истечении срока действия текущего). Токен доступа имеет 24-часовой срок действия; токен обновления имеет 7-дневный срок действия. Такой дизайн означает, что пользователю не нужно каждый день повторно вводить свой адрес электронной почты, но украденный токен доступа имеет ограниченное окно неправильного использования.

2.2.7 HikariCP: объединение соединений с базой данных

HikariCP - это высокопроизводительный пул соединений JDBC для приложений Java / Kotlin. Пул подключений - это кэш подключений к базе данных, которые повторно используются при выполнении запросов, а не создаются и уничтожаются для каждого запроса.

Открытие соединения с базой данных включает в себя установление связи по протоколу TCP с сервером базы данных, аутентификацию и создание сеанса на

стороне сервера - обычно это занимает 20-100 миллисекунд. Для конечной точки API, которая выполняет несколько запросов к базе данных, эти накладные расходы на каждый запрос увеличили бы время отклика на сотни миллисекунд.

HikariCP поддерживает пул предварительно открытых подключений (по умолчанию: 10 подключений). Когда приложению требуется запросить базу данных, оно заимствует соединение из пула (почти мгновенно), выполняет запрос и возвращает соединение с пулом. Пул настроен с максимальным количеством подключений (чтобы избежать перегрузки базы данных) и таймаутом подключения (для быстрого сбоя, если все подключения заняты).

2.2.8 Мультиплатформенность Kotlin (KMP): Кроссплатформенная мобильная разработка

Мультиплатформенность Kotlin (KMP) - это технология, разработанная компанией JetBrains, которая позволяет компилировать код Kotlin для нескольких целевых платформ из одной кодовой базы. Вместо написания отдельных реализаций в Swift для iOS и Kotlin для Android, KMP позволяет разработчику написать общую бизнес-логику в Kotlin один раз и скомпилировать ее как в JVM (для Android), так и в машинный код (для iOS через Kotlin / Native).

“Мультиплатформенная” модель различает:

- **commonMain:** общий код для всех платформ — бизнес-логика, модели данных, варианты использования, интерфейсы репозитория и сетевые клиенты
- **androidMain:** специфичные для Android реализации
- **iosMain:** специфичные для iOS реализации

Ключевой момент заключается в том, что код, зависящий от конкретной платформы, должен быть как можно меньше. В мобильном приложении ProductiveSocial клиент API, модели данных, логика синхронизации и бизнес-правила находятся в **commonMain**; только драйверы баз данных для конкретной платформы и точки интеграции пользовательского интерфейса находятся в наборах исходных текстов для конкретной платформы.

Почему KMP превосходит React Native или Flutter? Основная причина заключается в том, что KMP использует общий код с серверной частью: на те же классы модели данных, которые используются в серверных сервисах Kotlin, можно напрямую ссылаться в **commonMain** мобильного приложения. Класс данных задачи, определенный в совместно используемом модуле, имеет

идентичные имена и типы полей на сервере и клиенте, что устраняет класс ошибок, из-за которых сервер и клиент имеют разные определения одной и той же сущности. Кроме того, для команды разработчиков, работающих на Android, KMP кажется естественным, поскольку основной язык (Kotlin) и инструментарий (Gradle, Android Studio) уже знакомы.

Compose Multiplatform - это фреймворк пользовательского интерфейса, построенный поверх KMP. Он расширяет возможности Jetpack Compose (инструментария современного пользовательского интерфейса Google для Android) для работы на iOS и настольных компьютерах. Пользовательский интерфейс пишется один раз в Kotlin с использованием декларативной парадигмы Compose и компилируется для собственных компонентов пользовательского интерфейса на каждой платформе.

2.2.9 SQLDelight: Типобезопасный SQL для мультиплатформенной Kotlin

SQLDelight - это библиотека баз данных для мультиплатформенной Kotlin, которая генерирует типобезопасные API Kotlin из инструкций SQL. Вместо записи строк SQL во время выполнения (которые не проверяются компилятором и могут завершаться сбоем во время выполнения с непрозрачными ошибками), разработчик записывает SQL в .sq файлы, а SQLDelight генерирует функции Kotlin, которые выполняют эти запросы с полностью типизированными параметрами и возвращаемыми значениями.

Например, запрос задачи:

```
-- In tasks.sq
selectAllByUser:
SELECT id, sync_id, title, priority, completed, created_at
FROM tasks
WHERE user_id = :userId
ORDER BY created_at DESC;
```

Генерирует:

```
database.tasksQueries.selectAllByUser(userId = 42).executeAsList()
// returns List<SelectAllByUser> - a generated data class with typed fields
```

SQLDelight поддерживает как Android (используя SQLiteDriver от Android), так и iOS (используя NativeSqliteDriver) с одинаковыми определениями запросов и одинаковым сгенерированным API на обеих платформах. Это уровень локальной базы данных для механизма синхронизации в автономном режиме: все данные, созданные на мобильном устройстве, сначала записываются в SQLDelight, а конечная точка сетевой синхронизации вызывается в фоновом режиме.

2.2.10 Streamlit: быстрая разработка информационной панели

Streamlit - фреймворк Python с открытым исходным кодом для создания веб-приложений на основе скриптов Python без написания HTML, CSS или JavaScript. Разработчик пишет скрипт на Python, который вызывает потоковые функции (*st.title()*, *st.dataframe()*, *st.button()*) и Streamlit отображает их как интерактивные веб-компоненты в браузере.

Почему Streamlit для панелей мониторинга? Панели управления администратора и пользователя в ProductiveSocial являются операционными интерфейсами — они обеспечивают доступ к данным системы для отладки, мониторинга и пользовательских операций. Они не предназначены для того, чтобы быть основным продуктом, ориентированным на пользователя (как мобильное приложение КМР). Для этого варианта использования высокая скорость разработки Streamlit является решающим преимуществом: полнофункциональная панель администратора с навигацией, таблицами данных, формами и диаграммами может быть записана в одном файле Python объемом в несколько сотен строк.

Панели мониторинга взаимодействуют с серверными службами посредством HTTP-вызовов, используя библиотеку *запросов*, аутентифицируемую с помощью того же JWT, который использовал бы любой клиент API. Такой дизайн означает, что панели мониторинга являются настоящими клиентами API — они используют те же конечные точки и получают те же ответы, что и мобильное приложение, что делает их ценными интеграционными тестами в дополнение к операционным инструментам.

2.2.11 Коин: внедрение зависимостей для Kotlin

Внедрение зависимостей (DI) - это шаблон проектирования, в котором зависимости объекта предоставляются ему извне, а не создаются внутри. Вместо службы, создающей свое собственное подключение к базе данных и HTTP-клиент, эти зависимости создаются в другом месте и внедряются в конструктор службы.

Преимуществами являются тестируемость (в тестах реальные зависимости могут быть заменены *mocks*), конфигурируемость (разные среды могут внедрять разные реализации) и эксплицитность (график зависимостей объявлен в одном месте, а не разбросан по кодовой базе).

Koin - это облегченный DI-фреймворк для Kotlin. В отличие от Spring's DI (который использует отражение и обработку аннотаций), Koin использует Kotlin

DSL для объявления графика зависимостей. Это делает график читабельным, а обратную связь во время компиляции более четкой.

В ProductiveSocial каждый сервис Kotlin определяет свой модуль:

```
val appModule = module {
    single { DatabaseFactory(get()) }
    single { UserRepository(get()) }
    single { TaskRepository(get()) }
    single { HabitRepository(get()) }
    single { SyncService(get(), get(), get()) }
    single { HttpClient(CIO) { install(ContentNegotiation) { json() } } }
}
```

single означает одноэлементный экземпляр, который создается и используется повторно. *получить()* означает “разрешить из контейнера”. Когда Koin создает *SyncService*, он автоматически разрешает три необходимых ему репозитория из контейнера.

2.2.12 Ollama: локальный вывод LLM

Ollama - это инструмент с открытым исходным кодом для запуска больших языковых моделей локально на компьютере разработчика. Он предоставляет простой интерфейс командной строки для загрузки и запуска моделей (`ollama run llama3.2`) и предоставляет их через REST API, совместимый с форматом OpenAI API. Такая совместимость означает, что любой клиент, написанный на основе OpenAI API, может переключиться на Ollama путем изменения базового URL-адреса без каких-либо других изменений кода.

Почему Оллама? Во время разработки и тестирования использование облачного провайдера LLM (Anthropic или OpenAI) для каждого тестового запроса повлечет за собой затраты на API и приведет к задержке в сети. Ollama позволяет выполнять один и тот же вывод LLM локально, конфиденциально и бесплатно. Используемая модель (llama3.2 с параметрами 3B) достаточно мала, чтобы комфортно работать на процессоре современного ноутбука или графическом процессоре серии M.

Ограничение заключается в том, что локальный вывод Ollama недоступен из облачных сервисов (таких как служба выставления счетов с развертыванием рендеринга). Чтобы восполнить этот пробел в производственном тестировании, локальный экземпляр Ollama доступен через туннель ngrok (описано в следующем разделе).....

2.2.13 Ngrok: безопасное туннелирование для местных служб

Ngrok - это инструмент, который создает безопасный общедоступный URL-адрес HTTPS, который перенаправляет трафик службе, запущенной на локальном компьютере. Когда ngrok запущен и направлен на localhost: 11434 (порт Ollama), он создает туннель, через который любая машина в Интернете может отправлять запросы к локальному экземпляру Ollama. В

Переменная среды OLLAMA_BASE_URL биллинговой службы, размещенной на рендеринге, имеет значение ngrok HTTPS URL. Когда пользователь запускает анализ, биллинговая служба отправляет запрос на вывод по этому URL-адресу, который направляется через ngrok's infrastructure to the developer's machine, processed by Ollama, и возвращается ответ.,,

Такая схема подходит только для проверки концепции, когда один разработчик тестирует систему. Для производственной системы с несколькими пользователями, делающими одновременные запросы, потребуется надлежащая облачная модель, обслуживающая инфраструктуру.

Клиент Ollama биллинговой службы включает в себя две специальные конфигурации для ngrok с ngrok

```
headers = {
    "Content-Type": "application/json",
    "ngrok-skip-browser-warning": "true" # Skip ngrok's browser warning page
}
response = requests.post(
    f"{ollama_base_url}/api/chat",
    json=payload,
    headers=headers,
    allow_redirects=True, # Follow ngrok's redirect
    timeout=120
)
```

2.2.14 Визуализация: платформа облачного развертывания

Render - это облачная платформа для размещения веб-сервисов, баз данных и статических сайтов. Он предоставляет управляемую среду развертывания, в которой разработчик загружает код в репозиторий GitHub, а Render автоматически создает и развертывает его.

Для ProductiveSocial каждый сервис представляет собой отдельный веб-сервис визуализации, подключенный к собственному репозиторию GitHub. Процесс развертывания заключается в следующем:

1. Разработчик отправляет коммит в репозиторий сервиса на GitHub
2. Рендеринг распознает нажатие с помощью webhook
3. Render создает образ Docker из файла Docker в репозитории
4. Рендеринг развертывает новое изображение, заменяя запущенный контейнер

Бесплатный уровень Render обеспечивает достаточную емкость для развертывания с проверкой концепции. Заметное ограничение, описанное в разделе 2.8, заключается в том, что сервисы бесплатного уровня автоматически отключаются после 15 минут бездействия и при следующем запросе на них налагается штраф за "холодный запуск".

2.2.15 Neon PostgreSQL: Бессерверная управляемая база данных

Neon - бессерверная платформа PostgreSQL. Как и Render, она предоставляет базы данных PostgreSQL, не требуя от разработчика управления серверами баз данных. Для каждой базы данных автоматически предоставляется пул подключений, восстановление на определенный момент времени и ветвление (возможность создания копии базы данных в определенный момент времени для тестирования или разработки).

Каждый из четырех сервисов ProductiveSocial имеет свой собственный кластер баз данных Neon. Переменная среды DATABASE_URL для каждой службы соответствует стандартному формату строки подключения PostgreSQL:

```
postgresql://user:password@ep-xxx.us-east-2.aws.neon.tech/selfmanager?sslmode=require
```

Бесплатный уровень Neon подходит для рабочих нагрузок при разработке и тестировании. Для работы с реальными пользователями платный уровень будет необходим для гарантированной доступности и ограничений на подключение.

2.2.16 Сопрограммы Kotlin: структурированный параллелизм

Сопрограммы Kotlin - это языковая функция для написания асинхронного кода, который выглядит и ведет себя как последовательный код. Сопрограмма - это вычислительная единица, которая может быть приостановлена (в ожидании

ответа сети, результата базы данных или таймера) без блокирования потока, в котором она выполняется.

Проблема с потоками. В традиционном синхронном коде каждая блокирующая операция (сетевой вызов, запрос к базе данных) связывает поток на время ожидания. Для сервера, обрабатывающего 100 одновременных запросов, каждый из которых выполняет 3 сетевых вызова по 200 мс каждый, наивной реализации требуется 100 потоков только для запросов в процессе выполнения, что требует больших затрат памяти и затрат на планирование.

Решение сопрограммы. Сопрограммы имеют небольшой вес. Тысячи сопрограмм могут выполняться в небольшом пуле потоков. Когда сопрограмма приостанавливается (ожидая ответа), поток, в котором она была запущена, освобождается для выполнения других сопрограмм. Когда поступает ответ, сопрограмма возобновляется в любом доступном потоке.

Для аналитической службы ProductiveSocial сопрограммы предоставляют два особых преимущества: четыре вышестоящих вызова (задачи, привычки, процедуры, статистика таймера) запускаются как параллельные сопрограммы и ожидают вместе, сокращая общую задержку с суммы всех четырех раз ответа до максимальной для любого отдельного времени ответа. И если какой-либо вышестоящий вызов завершается неудачей, модель структурированного параллелизма гарантирует, что сбой обрабатывается в правильном месте без утечки сопрограмм.

2.2.17 Gradle и Shadow JAR

Gradle - это система сборки, используемая для всех сервисов Kotlin. FatJar (также называемый shadow JAR или uber JAR) представляет собой отдельный файл .jar, который содержит код приложения плюс все его зависимости, объединенные вместе. Обычно файл JAR содержит только собственные скомпилированные классы приложения и ожидает, что его зависимости будут находиться в classpath отдельно. Файл fat JAR является автономным: он может быть выполнен с помощью одной `java -jar app.jar` команды без дополнительной настройки пути к классу.

Задача Gradle shadowJar (из плагина `com.github.johnrengelman.shadow`) создает этот артефакт. Результирующий файл для каждой службы Kotlin имеет размер примерно 20-30 МБ (код приложения плюс Ktor, Exposed, HikariCP и все транзитивные зависимости).

Такой выбор дизайна упрощает развертывание: Dockerfile для каждой службы Kotlin просто копирует предварительно созданный JAR-файл и запускает его. В

сборке Docker нет этапа компиляции — образ создается за секунды, а не за минуты.

2.2.18 Сводная таблица

Technology	Role	Why Chosen
Kotlin	Language for 3 backend services + mobile	Null safety, coroutines, multiplatform
Ktor	HTTP server/client framework	Lightweight, coroutine-native, plugin-based
Python	Language for billing service	Native ML ecosystem (sklearn, LLM SDKs)
FastAPI	HTTP framework for billing	Async-first, automatic OpenAPI docs, Pydantic
PostgreSQL	Database for all services	Reliability, JSON support, advanced indexing
Neon	Managed PostgreSQL in cloud	Free tier, no ops overhead
Docker	Containerization	Environment consistency, portability
Colima	Docker runtime on macOS	Lightweight, free, Docker-compatible
JWT	Authentication tokens	Stateless, cross-service verifiable
KMP	Mobile cross-platform framework	Single Kotlin codebase for Android + iOS
Compose Multiplatform	Shared UI for mobile	Unified UI codebase with KMP
SQLDelight	Mobile local database	Type-safe, KMP-compatible, offline storage
Streamlit	Admin and user dashboards	Rapid Python dashboard development
Koin	Dependency injection	Lightweight, Kotlin DSL, no reflection
Ollama	Local LLM inference	Free, private, development-friendly
Ngrok	Local→cloud tunnel	Exposes local Ollama to Render
Render	Cloud hosting	Free tier, git-push deploy
HikariCP	JDBC connection pooling	High performance, widely used
Gradle Shadow	Fat JAR build	Self-contained deployment artifact

2.3 Стратегия обработки данных: данные, генерируемые приложением

Одним из наиболее существенных отличий подхода ProductiveSocial ML от обычного проекта машинного обучения является стратегия обработки данных. В этом проекте не используются внешние наборы данных. Ни один общедоступный набор данных о производительности (Kaggle Time Management Insights, PMData или аналогичный) не использовался для обучения, проверки или оперативного построения. Конвейер подготовки данных построен не был. Разработка функций в общепринятом смысле этого слова не проводилась.

Это обдуманное архитектурное решение, а не ограничение, и понимание того, почему это происходит, требует понимания фундаментальной разницы между двумя основными подходами к анализу на основе ML:

Подход А: контролируемое прогнозирование на основе данных о населении.

Подготовьте модель на основе набора данных о производительности многих пользователей с метками достоверности (например, “оценка производительности этого пользователя сегодня составила 7,2 / 10 на основе самоотчета”). Используйте модель для прогнозирования показателей производительности или результатов для новых пользователей на основе их моделей активности. Этот подход требует наличия маркированного набора данных, базовых определений достоверности, разработки функций, инфраструктуры обучения и настройки гиперпараметров. Это неприменимо на данном этапе разработки ProductiveSocial, поскольку такого набора данных не существует.

Подход В: Контекстуальное обоснование данных, специфичных для конкретного пользователя. На этапе анализа соберите полные данные об активности пользователя и передайте их языковой модели, которая рассуждает об этом, используя свои предварительно подготовленные знания о моделях производительности человека. Никакого обучения не требуется; способность модели рассуждать зависит от ее предварительной подготовки, а качество анализа зависит от богатства предоставленного контекста. Этот подход применим непосредственно на любом этапе разработки, не требует предварительных пользовательских данных и естественным образом масштабируется в зависимости от объема доступных данных (пользователь с 30-дневной историей получает лучший анализ, чем пользователь с 3-дневной).

ProductiveSocial использует подход В. Это не компромисс; это правильный подход для текущей стадии проекта. Подход А становится жизнеспособным — и предпочтительным для определенных задач прогнозирования — после накопления реальных данных о поведении пользователей в больших масштабах. Архитектура явно сохраняет путь для этого перехода: путь вывода sklearn биллинговой службы, реестр моделей, и конечная точка загрузки .pkl

разработаны специально для размещения обученных моделей без каких-либо изменений в более широкой системе.

Собственная генерация данных для тестирования. Вместо использования внешних наборов данных тестовый профиль пользователя был создан вручную, чтобы представить реального и разнообразного пользователя: три проекта, охватывающие профессиональную работу, личностное развитие и приобретение навыков; восемь задач с различными приоритетами, состояниями завершения и затратами времени; шесть привычек, охватывающих как позитивное поведение для формирования, так и негативное поведение для прекращения; четыре процедуры, представляющие наиболее распространенные временные якоря продуктивного дня; и 32 фокусных сеанса продолжительностью в две недели для всех типов сущностей. Эти тестовые данные служат как проверочным набором данных, так и демонстрацией аналитических возможностей системы.

2.4 Конвейер анализа на основе LLM

Конвейер анализа аналитической службы analytics service является центральным техническим вкладом в интеграцию ML этого проекта. Он реализует шаблон генерации с расширенным извлечением (RAG), в котором на этапе “извлечения” извлекаются данные пользователя в реальном времени из внутренних служб, а не из векторной базы данных.

Когда пользователь отправляет запрос на анализ (*POST /api/v1/analytics с типом анализа и идентификатором модели*), *AnalyticsService* выполняет следующую последовательность действий:

Шаг 1: Сбор пользовательского контекста. Выполняются три параллельных HTTP-вызова с использованием внутреннего служебного ключа:

- *ПОЛУЧИТЬ /internal/users/{userId}/задачи* → selfmanager возвращает список задач пользователя с именами, приоритетами, состояниями завершения и значениями затраченного времени
- *ПОЛУЧИТЬ /internal/users/{userId}/привычки* → selfmanager возвращает привычки с именами, типами (Запуск / Завершение), повторяемостью и значениями затраченного времени
- *GET /internal/users/{userId}/подпрограммы* → selfmanager возвращает подпрограммы с именами и рекуррентностью
- *ПОЛУЧИТЬ /внутренние /пользователи /{идентификатор пользователя}/статистика* → таймер возвращает агрегированную статистику сеанса

Эти вызовы выполняются с неблокирующей архитектурой: если `selfmanager` или `timer` недоступны, для соответствующих данных устанавливается пустой набор, и анализ продолжается с любыми доступными данными. Сбой выставления счета (этап вывода LLM) восстановлению не подлежит — пользователю он выдается как ошибка.

Шаг 2: Построение структурированной полезной нагрузки. Собранные данные собираются в структурированную карту, которая станет контекстом LLM.

Шаг 3: Вывод LLM с помощью выставления счетов. Структурированная полезная нагрузка сериализуется в JSON и отправляется в *psocial_billing* через *POST /api/v1/internal/predict* с заголовком *X-Internal-Key*. Служба выставления счетов получает полезную нагрузку, проверяет наличие у пользователя достаточного количества кредитов, создает ОЖИДАЮЩУЮ запись прогноза, передает данные выбранному поставщику LLM и возвращает результат вывода вместе с начисленными кредитами.

Шаг 4: Сообщите о постоянстве. Текст аналитической информации, извлеченный из ответа о выставлении счета, сохраняется в *таблице analytics_reports* с идентификатором пользователя, типом анализа, идентификатором модели, стоимостью кредита и отметкой времени. Результат возвращается пользователю.

2.5 Оперативные виды проектирования и анализа

Качество анализа LLM в значительной степени определяется спецификой и уместностью запроса. Для каждого из шести типов анализа была разработана отдельная подсказка, позволяющая сфокусировать рассуждения модели на конкретном вопросе, который задает пользователь.

PRODUCTIVITY_SUMMARY использует самую широкую подсказку:

“На основе задач, привычек, распорядка дня пользователя и сеансов фокусирования предоставьте краткое описание производительности. Выделите то, что прошло хорошо и что требует внимания”.

Это приглашение намеренно носит открытый характер, позволяя модели синтезировать весь доступный контекст в общий обзор. Ожидается, что модель выявит закономерности, такие как: множество невыполненных высокоприоритетных задач, несмотря на значительное время сосредоточения (потенциальная проблема с расстановкой приоритетов), привычки, последовательно выполняемые в дни с завершенными сеансами сосредоточения, но не в дни без них (корреляция привычки и сосредоточенности), или рутины с

низкой приверженностью по сравнению с простыми привычками (структурные тренировки в рутине).

При ПРИОРИТИЗАЦИИ ЗАДАЧ используется более директивная подсказка:

“Учитывая незавершенные задачи пользователя и затраченное на данный момент время, предложите список приоритетных задач на сегодня. Будьте конкретны и осуществимы”.

Это приглашение фокусирует модель на подмножестве незавершенных задач и просит ее составить ранжированный список, а не описание. Ожидается, что в модели будут учитываться метки приоритетов, близость крайнего срока и уже затраченное время.

HABIT_INSIGHTS фокусируется на поведенческих паттернах:

“Проанализируйте привычки пользователя. Какие из них постоянны? Какие подвержены риску? Какое изменение окажет наибольшее влияние?”

Последний вопрос — “какое изменение оказало бы наибольшее влияние?” — намеренно задан в единственном числе. Это заставляет модель расставлять приоритеты, а не перечислять все возможные улучшения, обеспечивая более действенный результат.

ROUTINE_OPTIMIZATION фокусируется на эффективности процесса:

“Проанализируйте процедуры пользователя и затраченное время. Как их можно сделать более эффективными? Предложите конкретные улучшения”.

Процедуры являются наиболее структурированными из отслеживаемых объектов (упорядоченные шаги с длительностью), и это приглашение запрашивает у модели информацию об их структуре, а не просто об их соблюдении.

Запрос WEEKLY_TIMER_SUMMARY является наиболее ограниченным:

“На основе данных сеанса Pomodoro пользователя создайте краткое еженедельное резюме по фокусу (3-4 предложения). Укажите общее время фокусировки, частоту завершения сеанса, на каком типе объекта они сосредоточились больше всего и их наиболее продуктивные шаблоны. Сохраняйте теплый тон и нацеленность на будущее. Завершите одним практическим предложением на предстоящую неделю”.

Явное ограничение по длине (3-4 предложения), перечисленные требования к контенту и тональность инструкции отражают уроки, извлеченные из раннего тестирования, в ходе которого модель выдавала либо слишком краткие резюме,

в которых отсутствовали ключевые данные, либо слишком длинные описания, которые не были прочитаны пользователями полностью.

ПОЛЬЗОВАТЕЛЬСКИЙ интерфейс принимает предоставленную пользователем подсказку, которая используется как есть:

“текст, предоставленный пользователем”

Этот тип анализа является наиболее гибким и наименее предсказуемым по качеству вывода, поскольку качество запроса полностью зависит от пользователя. Это полезно для вопросов, которые не соответствуют предопределенным типам, и для опытных пользователей, которые хотят задать конкретные вопросы о своих данных.

Системная подсказка, хранящаяся в реестре моделей для модели Ollama по умолчанию, обеспечивает общую структуру для всех типов анализа:

“Вы опытный тренер по продуктивности. У вас есть доступ к задачам, привычкам, распорядку дня пользователя и данным сеанса фокусировки. Основываясь на этих данных, проведите глубокий, действенный и персонализированный анализ ”.

Это системное приглашение определяет роль модели и устанавливает ожидания от стиля вывода (содержательный, действенный, персонализированный), не ограничивая содержание какого-либо конкретного типа анализа.

2.6 Показатели оценки

Для обеспечения качества анализа LLM:

Основной показатель - это оценка человека по шкале от 1 до 5 баллов по трем измерениям:

- Актуальность: обращается ли анализ к конкретному вопросу, на который призван ответить данный тип анализа?
- Специфичность: ссылается ли анализ на конкретные элементы из данных пользователя (например, называет реальные задачи или привычки), а не делает общие наблюдения?
- Возможность принятия мер: дает ли анализ хотя бы одно конкретное предложение, на основании которого пользователь мог бы действовать?

Сводный балл - это среднее значение по трем измерениям. Базовый уровень, с которым сравниваются результаты LLM, представляет собой сводку на основе правил, сгенерированную без вывода LLM — простой текст, заполненный

шаблоном, в котором указаны подсчеты и средние значения без интерпретации (например, “У вас 8 задач, 3 выполнены. Ваша самая распространенная привычка - [название]. Вы потратили на фокус-сессии 310 минут.”).

Для корректности синхронизации:

- Полнота синхронизации: Процент записей в локальном тестовом наборе данных, которые корректно отображаются в базе данных сервера после операции синхронизации
- Идемпотентность: количество повторяющихся записей, созданных при повторной отправке одного и того же запроса на синхронизацию (целевое значение: 0)

Для кредитного учета:

- Точность баланса: $balance_before - cost_per_prediction = balance_after$ проверено для каждого прогноза в тестовом наборе

Для получения информации о доступности услуги:

- Задержка холодного запуска: время от первого HTTP-запроса до первого успешного ответа при рендеринге (службы JVM)
- Задержка прогрева: время для последующих запросов после прогрева сервиса
- Постепенное ухудшение: поведение аналитики, когда selfmanager или таймер временно недоступны

2.7 Анализ результатов

В следующей таблице описаны все эксперименты, проведенные на этапе валидации.

#	Тестовый	базовый	показатель	Результат	закл ^ю чени ^я
1	Межсервисная проверка подлинности JWT: token selfmanager принят analytics	N / A (архитектурные требования)	Токен принят / отклонен	Токен принят analytics, таймер; для недействительных токенов возвращает ^{ся} правильный 401.	Совместное использование JWT между сервисами реализовано корректно.
2	Приветственный кредитный депозит: новый пользователь получает 100 кредитов при первом вызове predict.	Нет предварительного баланса (0 кредитов).	balance_after при первой транзакции	100 кредитов вносятся автоматически; транзакция депозита создана.	Отложенная инициализация работает правильно.
3	Точность списания кредита: balance_before - стоимость balance_after	Начальный баланс: 100 кредитов; стоимость модели: 5 кредитов;	Математическое равенство	Баланс после прогнозирования: 95 кредитов; транзакция показывает сумму: -5, balance_after: 95	Арифметика кредитов верна
4	Идемпотентность синхронизации: один и тот же запрос отправлен дважды	дубликатов не ожидается	Количество повторяющихся записей	При повторной отправке возвращены те же idMappings	Идемпотентность через ClientID работает

#	Тестовый	базовый	показатель	Результат	закл ^ю чени ^я
				с 0 созданными новыми записями	корректно
5	Полнота синхронизации: все 32 сеанса синхронизированы после симулированного отключения	32 сеанса в тестовом наборе данных	% записей в базе данных сервера	присутствует 32/32 сеанса; выполнено 55/55 привычек; все 8 задач	Синхронизация для тестируемого набора данных завершена
6	PRODUCTIVITY_SUMMARY quality (LLM vs. базовый уровень)	Основанный на правилах: “У вас 8 задач, 3 выполнены ..” (оценка: 1,5 / 5)	Оценка персонала а 1-5 баллов (среднее значение релевантности, специфичности, возможности применения)	Балл LLM: 4,2 / 5; названы конкретные задачи и привычки; выявлена корреляция фокус-привычка.	Уровень LLM существенно превосходит базовый уровень, основанный на правилах.
7	Качество HABIT_INSIGHTS (уровень LLM по сравнению базовый уровень)	Основанный на правилах: “У вас есть 6 привычек. 5 - это начальный тип ”. (оценка: 1.0 / 5)	Оценка оценки персонала а 1-5	Оценка LLM: 3.9 / 5; выявлены привычки, подтвержденные риску, и предложено одно конкретное изменение.	LLM дает полезную информацию о поведении
8	Качество WEEKLY_TIMER_SUMMARY (LLM против базовый)	На основе правил: “У вас было 32 сеанса, 31	человека ивала счетом 1-	ЛЛМ оценка: 4.0/5; покрыты	ограниченны подсказут формат

#	Тестовый уровень)	базовый завершен”. (оценка: 1.5/5)	показател ь	Результат	заклучени я
			5	завершения которой, юридическо е лицо тип поломки, действенны е следующей неделе предложени е	производят последоват ельно, структурир овано выход
9	изящные деградации: аналитика, когда таймер недоступна	анализ совсем	не анализа завершае тся / завершае тся	анализ завершения с Pomodoro блок исключить из контекста	таймера неудачи неблокиру ющий как
10	услуги холодного пуска на отрисовку (JVM), на	Н/д	время выхода первой успешног о ответа	~52 секунды среднем для JVM, услуг; ~8 секунд для Python биллинг	для JVM холодный пуск является основным задержки узкое место на уровне бесплатног о пользован ия
11	искусству предсказания: нет кредитных вычета	остаток: 95 кредитов на	баланс не изменилс я после неудачно го вывода	баланса оставалось 95 после имитации ошибки поставщика ; искусству предсказани	не предсказан ия правильно освобожда ются от выставлен ия счета

#	Тестовый	базовый	показатель	Результат	заклучения
				я записи, без оплаты сделка	
12	админ залог: пользователь кредиты добавлено ский баланс: через внутренний 95 кредитов API,		balance_a fter на депозит	после внесения 50 остаток: 145; депозитные операции записываются	админ кредит менеджмент работает правильно

Пример вывода LLM для PRODUCTIVITY_SUMMARY (сокращенный для отчета):

Входной контекст (обобщенный): 8 задач (3 выполнены: 2 высокоприоритетных, 1 средняя; 5 незавершенных: 3 высокоприоритетных, 2 средних), 6 привычек (4 выполненных ежедневно на прошлой неделе, 2 непоследовательных — оба типа начальных), 4 рутины (Утренняя: соблюдение 85%, глубокая работа: 60%, остальные ниже), 31 сеанс Pomodoro (всего 310 рабочих минут, 94% выполнения, в основном связано с задачами).

Результат LLM (PRODUCTIVITY_SUMMARY через llama3.2):

“Уровень выполнения ваших задач демонстрирует четкую закономерность: высокоприоритетные задачи выполняются, но ваше отставание в виде 3 незавершенных высокоприоритетных задач говорит о том, что вы, возможно, устаете к концу дня, прежде чем приступите к ним. Рассмотрите возможность предварительной загрузки их в начале дня, особенно учитывая, что данные о сеансе фокусировки показывают, что самый высокий процент завершения у вас до 14: 00. Ваш четкий рабочий график с соблюдением 60%—ного графика является вашим самым большим препятствием - разрыв между запланированной структурой и фактическим выполнением предполагает, что его, возможно, потребуется сократить или упростить. Две привычки — [Привычка А] и [Привычка В] — не были выполнены в те дни, когда количество сеансов фокусировки было нулевым, что указывает на то, что эти привычки могут зависеть от импульса. Проведение короткого сеанса с Помидорами первым делом с утра может помочь им закрепиться”.

Базовый вывод на основе правил (тот же ввод):

“У вас есть 8 задач, 3 из которых выполнены и 5 находятся на рассмотрении. Ваша самая активная привычка - [Привычка С] с 7 завершениями. У вас есть 4 рутины. Вы завершили 31 фокус-сессию общей продолжительностью 310 минут”.

Качественная разница между двумя результатами очевидна. Выходные данные LLM ссылаются на конкретные точки данных, определяют междоменную корреляцию (привычки и сеансы фокусирования), называют конкретные изменения, требующие принятия мер, и предоставляют конкретное обоснование. Базовые выходные данные точны, но не дают никакой информации, кроме пересчета входных данных.

2.8 Анализ ошибок и извлеченные уроки

В ходе разработки и тестирования было обнаружено несколько проблем, которые стоит задокументировать, поскольку они влияют на окончательный дизайн системы.

Проблема 1: запрос LLM не получил структурированные данные (решен)

В ранней версии интеграции служба выставления счетов получала запрос аналитики, но структурированные данные пользователя (задачи, привычки, распорядок дня, статистика Pomodoro) неправильно передавались в запрос LLM. Модель получала только текст запроса для конкретного типа без каких-либо пользовательских данных, выдавая общий ответ без ссылки на фактический контекст пользователя. Основной причиной было несоответствие между ожидаемой структурой JSON в *PredictionService* биллинговой службы структурой, отправляемой биллингом клиентом аналитической службы.....

Решение: Конструктор запросов LLM биллинговой службы был обновлен для правильного извлечения и форматирования всех полей из карты *inputData*, а структура полезной нагрузки аналитической службы была приведена в соответствие с ожиданиями биллинговой службы. Исправление было проверено путем подтверждения того, что в выходных данных LLM указаны конкретные задачи и привычки пользователя.

Проблема 2: холодный запуск JVM на уровне без рендеринга

Бесплатный уровень Render отключает сервисы после 15 минут бездействия. Для трех сервисов на основе JVM (*selfmanager*, *timer*, *analytics*) первый запрос после спиндауна с задержкой холодного запуска составляет ~ 50 секунд. Для службы выставления счетов Python холодный запуск составляет примерно 8 секунд. Это было устранено на пользовательских информационных панелях путем отображения предупреждающего сообщения при входе в систему, указывающего, что для пробуждения служб при первом использовании может потребоваться до 60 секунд.

Проблема 3: для рендеринга Ollama требуется туннель Ngrok

Локальный экземпляр Ollama (работающий под управлением llama3.2) недоступен напрямую из служб, размещенных на рендеринге. Чтобы включить производственное тестирование, локальный экземпляр Ollama предоставляется через туннель HTTPS ngrok HTTPS tunnel, and the billing service's *OLLAMA_BASE_URL* environment variable on Render points to the ngrok. Клиент Ollama биллинговой службы был обновлен, чтобы включить заголовок *ngrok-skip-browser-warning* и следить за перенаправлениями, оба из которых

необходимы для корректной работы туннеля ngrok. Эта настройка подходит для проверки концепции, но не для производственного использования - при производственном развертывании будет использоваться размещенный в облаке экземпляр Ollama или переключение на размещенного провайдера (Anthropic или OpenAI).

2.9 Краткое изложение концепции

Текущая реализация ProductiveSocial представляет собой работающее подтверждение концепции основного ценностного предложения: единой платформы для повышения производительности, в которой все модули имеют общую идентификацию пользователя, а центральный уровень анализа искусственного интеллекта имеет одновременный доступ к междоменным поведенческим данным.

Были полностью продемонстрированы следующие возможности:

1. Пользователь может быть идентифицирован с помощью одного адреса электронной почты и получить JWT, который действителен для всех трех пользовательских сервисов (selfmanager, timer, analytics).
2. Пользователь может создавать задачи, привычки и процедуры в selfmanager и отслеживать их с помощью selfmanager API. Эти объекты немедленно становятся доступны службе аналитики через внутренний API.
3. Пользователь может создавать и завершать сеансы Pomodoro в таймере, необязательно связанные с задачами, привычками или рутинами. Затраченное время автоматически записывается обратно в связанный объект в selfmanager. Анализ шаблонов фокусировки вычисляется на основе истории сеанса.
4. Пользователь может запросить анализ искусственного интеллекта любого из шести типов. Служба аналитики собирает полный контекст производительности пользователя из selfmanager и timer, передает его в службу выставления счетов, которая вызывает LLM, начисляет кредиты и возвращает информацию о производительности. Отчет сохраняется и отображается пользователю.
5. Управление кредитами осуществляется прозрачно. При каждом прогнозировании стоимость модели списывается с баланса пользователя и регистрируется транзакция. Недостаточное количество кредитов блокирует анализ до того, как будет сделан какой-либо вызов LLM.

6. Панели мониторинга администратора и пользователя предоставляют полные интерфейсы для вышеуказанных возможностей, не требуя от пользователя никаких прямых вызовов API.

Что еще не включает в себя это доказательство концепции:

- Мобильное приложение производственного уровня (приложение КМР является каркасным, но не полнофункциональным)
- Собственные обученные модели ML (система использует запросы LLM для всего анализа)
- Социальные функции (планируемые для последующего этапа разработки)
- Модуль журнала или заметок (отложен до завершения MVP)
- Масштабируемый LLM-хостинг (текущее развертывание рендеринга использует личный экземпляр через ngrok)

Архитектура явно учитывает каждую из этих будущих возможностей без структурных изменений существующих сервисов.

Глава 3: Архитектура и реализация

3.1 Системные проектные решения и архитектурные принципы

Архитектура ProductiveSocial была разработана на основе набора четких принципов, которые управляют каждым важным решением в системе:

Принцип 1: Независимость сервиса выше удобства интеграции. Каждый сервис разработан таким образом, чтобы он функционировал корректно, даже когда другие сервисы недоступны. Таймер работает без selfmanager (сеансы могут быть автономными, а вызовы журнала времени запускаются и забываются). Selfmanager работает без таймера (управление задачами и привычками полностью функционально без данных Pomodoro). Аналитика изящно ухудшается, когда selfmanager или timer недоступны (она продолжает работу с любыми данными, которые получает). Биллинг работает без использования какой-либо другой службы (он только получает запросы и обрабатывает их). Этот принцип приносит в жертву некоторое удобство разработки (было бы проще иметь единую монолитную службу или жестко реагировать на ошибки восходящего потока) в обмен на систему, устойчивую к частичным сбоям.

Принцип 2: Единый источник истины для каждой области. Идентификационные данные пользователя принадлежат selfmanager (и общей регистрации пользователей). Данные о задачах, привычках и распорядке дня принадлежат selfmanager. Данные сеанса фокусировки принадлежат таймеру. Учетные записи и логические выводы принадлежат billing. Отчеты Analytics принадлежат analytics. Ни один сервис не дублирует данные, принадлежащие другому сервису — он извлекает данные с помощью внутренних вызовов API в то время, когда это необходимо. Это предотвращает проблемы с согласованностью, возникающие при репликации одних и тех же данных в нескольких сервисах.

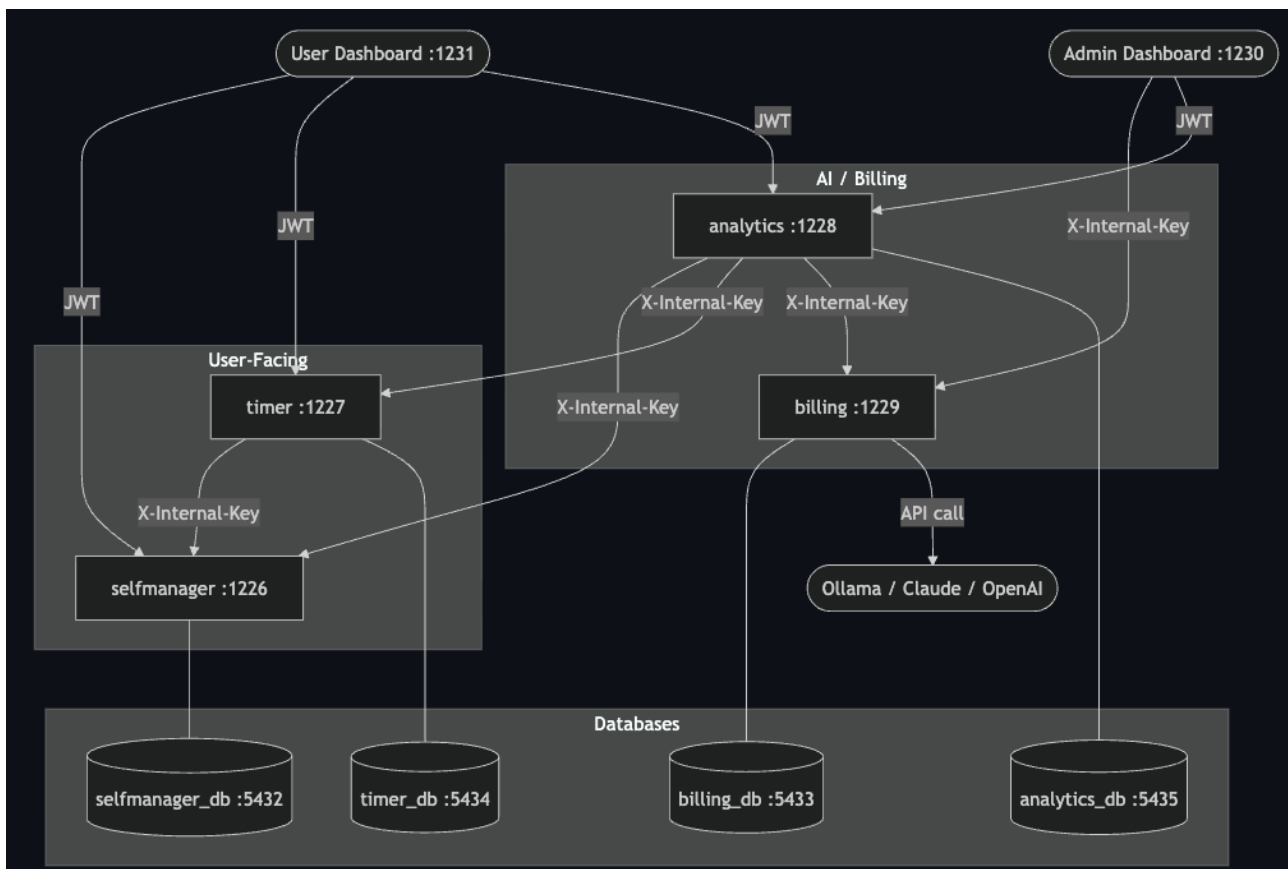
Принцип 3: Весь внешний доступ через аутентифицированные конечные точки, весь внутренний доступ через общий секрет. Граница между “обращением к клиенту” и “обменом услугами” проводится на уровне HTTP. Клиентские запросы содержат JWT в заголовке Authorization: Bearer. Запросы от услуги к услуге содержат общий внутренний ключ в заголовке X-Internal-Key. Биллинг не имеет никакого клиентского доступа — для всех его конечных точек требуется внутренний ключ. Это означает, что даже если URL-адрес биллинга каким-либо образом будет раскрыт, ни один клиент не сможет взаимодействовать с ним, не владея внутренним ключом, который хранится только в переменных окружения на сервере.

Принцип 4: Офлайн - первое владение данными. Мобильные клиенты владеют своими данными локально. Сервер является целью синхронизации, а не источником достоверности во время автономной работы. Для этого требуются назначенные клиентом UUID для всех объектов (чтобы клиент мог создавать записи, не дожидаясь ответа сервера), операции идемпотентной синхронизации (чтобы повторная попытка неудачной синхронизации никогда не приводила к повреждению данных) и явное отслеживание удалений (чтобы клиент знал, какие записи удалять из его локальной базы данных).

Принцип 5: Кредитная система как поведенческий стимул. Решение о внедрении кредитной системы для анализа ИИ служит цели, выходящей за рамки учета затрат. Делая стоимость каждого анализа видимой и вычитая ее из конечного баланса, система создает естественный стимул для пользователей использовать функции анализа сознательно, а не навязчиво. Это согласуется с философией платформы, направленной на снижение когнитивных издержек, а не на их увеличение. Приветственный кредитный депозит (100 кредитов при первом использовании) устраняет барьер для входа, а возможность администратора пополнять кредиты гарантирует, что доступ не будет ограничен оплатой для пользователей, которым это необходимо.

3.2 Архитектура микросервисов

Серверная часть ProductiveSocial состоит из микросервисов, каждый из которых имеет собственную базу данных, кодовую базу и конфигурацию развертывания.



Selfmanager – это служба идентификации и передачи данных о производительности. Это центральный узел данных платформы, которому принадлежит таблица *users*, на которую опирается вся система для идентификации пользователей.

Таймер – этому сервису принадлежит жизненный цикл сеанса pomodoro и вся аналитика focus.

Аналитика - наиболее сложный по составу сервис. Он организует вызовы трех вышестоящих служб, создает контекст LLM, вызывает выставление счетов, сохраняет результаты и предоставляет их пользователям и администраторам.

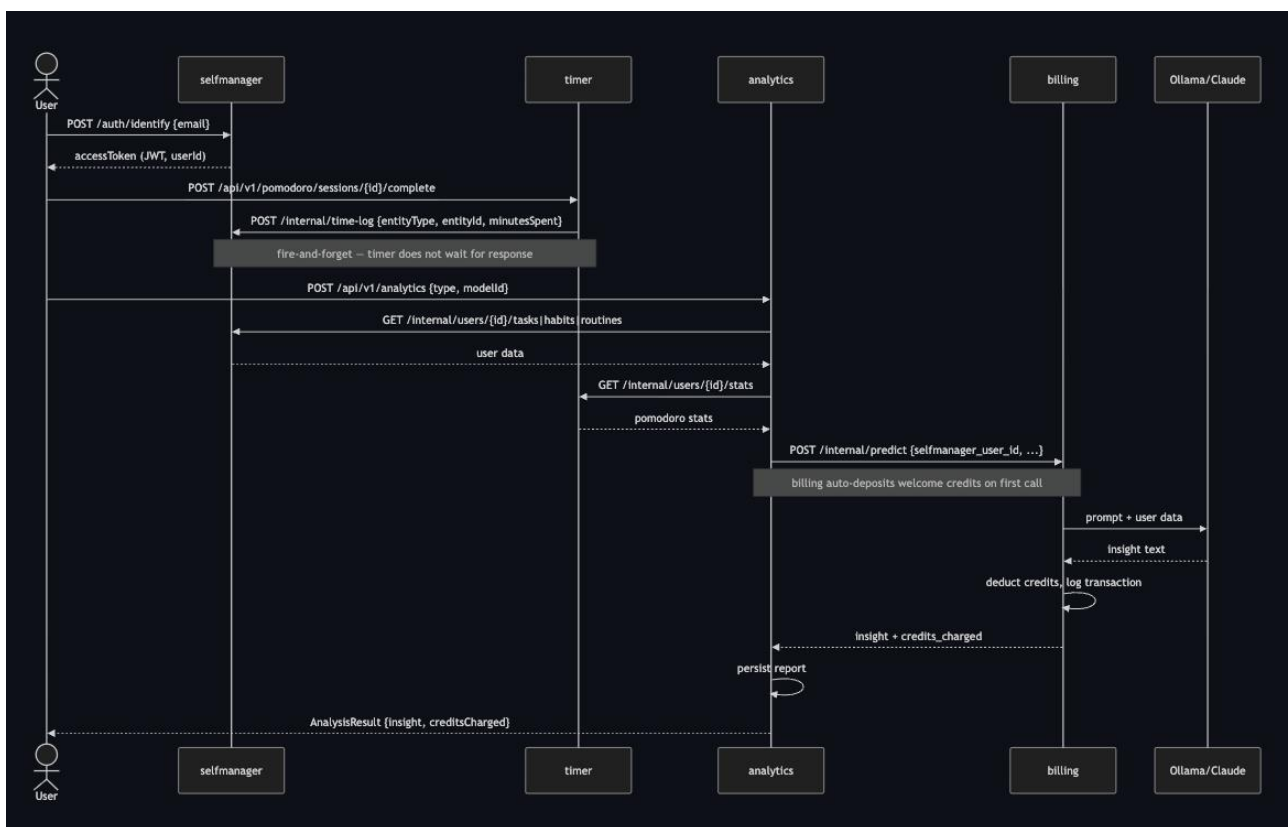
Биллинг - архитектурно самая простая, но технически самая важная услуга в системе. У него нет аутентификации JWT, нет общедоступных конечных точек и нет таблицы пользователей, он предоставляет доступ к службам ML, одновременно кредитует использование служб.

3.3 Аутентификация и межведомственная идентификация

Архитектура аутентификации решает нетривиальную проблему: как предоставить пользователю единую идентификацию, которая действительна в нескольких независимых сервисах, не требуя централизованного поставщика идентификационных данных или сложных протоколов обмена токенами.

Процесс входа в систему

Когда пользователь впервые открывает панель управления или мобильное приложение, он указывает только свой адрес электронной почты. Система использует модель идентификации “найди или создай”: если электронное письмо существует в базе данных, пользователь возвращается к своей существующей учетной записи; если нет, создается новая учетная запись. Здесь нет этапа проверки пароля и электронной почты — это сделано специально для этапа проверки концепции, где первоочередной задачей является демонстрация функциональности, а не безопасность производственного уровня.



Совместное использование JWT: один токен для всех сервисов

Все три пользовательских сервиса (selfmanager, timer, analytics) настроены с использованием одних и тех же переменных окружения JWT_SECRET, JWT_ISSUER, JWT_AUDIENCE и JWT_REALM. Поскольку все три службы разделяют эти значения, токен, подписанный selfmanager, проходит проверку в

службах timer и analytics. Службы не проверяют, кто выдал токен — они проверяют только действительность подписи (что так и есть, поскольку все они используют один и тот же секрет) и правильность формирования утверждений. Это фундаментальный механизм межведомственной аутентификации без централизованного поставщика удостоверений.

В пользовательской регистрации: каноническое целочисленное тождество

Более тонкая проблема идентификации заключается в обеспечении того, чтобы одному и тому же адресу электронной почты присваивалось одно и то же целое число user_id независимо от того, какая служба впервые сталкивается с пользователем. Это важно, потому что межведомственная привязка данных зависит от общего целочисленного ключа: если сеансы Pomodoro пользователя хранятся с идентификатором пользователя = 42 в базе данных timer, но их задачи хранятся с идентификатором пользователя = 99 в базе данных selfmanager, служба аналитики не может объединить эти два набора данных.

В UserRegistry это решается за счет того, что все службы, которым требуются идентификаторы пользователей, подключаются напрямую к таблице пользователей selfmanager в качестве канонического источника. Служба таймера имеет выделенный пул соединений HikariCP, указывающий на базу данных PostgreSQL от selfmanager (настраивается с помощью переменных окружения USER_REGISTRY_DB_*). Когда служба таймера получает запрос с JWT, она извлекает запрос по электронной почте и запускает:

```
INSERT INTO users (email, created_at)
VALUES (?, NOW())
ON CONFLICT (email) DO UPDATE SET email = EXCLUDED.email
RETURNING id
```

Эта ошибка upsert, или создает пользователя (возвращая новый идентификатор), или находит существующего пользователя (возвращая существующий идентификатор). Результатом всегда является канонический целочисленный идентификатор для этого электронного письма — тот же идентификатор, который selfmanager вернул бы для того же электронного письма. Предложение RETURNING id - это функция PostgreSQL, которая возвращает значение столбца ID для строки, которая была вставлена или обновлена.

Служба аналитики не использует UserRegistry напрямую — вместо этого она извлекает userId из утверждений JWT (которые selfmanager уже проверил и закодировал) и использует это значение напрямую. Это работает, потому что требование userId JWT было установлено selfmanager при выпуске токена, а selfmanager является авторитетным источником идентификаторов пользователей.

Единственным соединением между базами данных во всей системе является UserRegistry подключенное к базе данных selfmanager. Это явная и документированная зависимость, ее назначение узкое (один запрос: найти или создать пользователя по электронной почте), и у нее есть выделенный пул подключений, чтобы он не мешал нормальной работе таймера с базой данных.

Аутентификация по внутреннему ключу X: от сервиса к сервису

Для вызовов между службами (вызов аналитики selfmanager для получения пользовательских данных, вызов таймера selfmanager для обновления журнала времени, вызов аналитики billing для вывода данных) используется другой механизм аутентификации: предварительно разделяемый ключ в HTTP-заголовке X-Internal-Key.

Основная причина использования отдельного механизма для вызовов между службами вместо повторного использования JWTs заключается в том, что внутренние конечные точки предоставляют данные в совокупности (например, “все задачи для пользователя 42”) или выполняют операции, которые обходят бизнес-правила (например, начальные тестовые данные при выставлении счетов). Эти конечные точки должны быть доступны только доверенным службам, а не любому пользователю с действительным JWT. Использование отдельного заголовка с отдельным ключом делает эту границу явной.

Переменная среды INTERNAL_API_KEY имеет одинаковое значение для всех сервисов. Когда аналитика вызывает GET /internal/users/{userId}/задачи selfmanager, она включает X-Internal-Key: <ключ> в запрос. selfmanager проверяет этот заголовок перед выполнением обработчика. Если заголовок отсутствует или ключ не совпадает, ответ 401 Несанкционированный — независимо от того, содержит ли запрос также действительный JWT.

Внутренний ключ никогда не передается в исходный код, никогда не возвращается ни в одном ответе API и не документируется в общедоступном пользовательском интерфейсе Swagger. Он хранится исключительно в переменных окружения на сервере.

3.4 Модель данных и стратегия хранения

Каждая служба владеет своей собственной базой данных PostgreSQL со схемой, разработанной для конкретных шаблонов доступа к данным этой службы.

3.5 Реализация сервиса

Рабочий процесс разработки. Все сервисы Kotlin имеют одинаковую структуру проекта: многомодульный проект Gradle с серверным подмодулем, содержащим код приложения. Серверный модуль создается в fat JAR с помощью задачи shadowJar, которая объединяет все зависимости. Dockerfile для

каждого сервиса Kotlin копирует предварительно созданный файл fat JAR, а не создает его в контейнере, что сокращает время сборки Docker до 10 секунд. Служба выставления счетов Python использует одномодульное приложение FastAPI с alembic для миграции базы данных (заменено в текущей настройке разработки на SQLAlchemy create_all(), вызываемое при запуске, с необязательным флагом RESET_DB= true для сброса схемы).

Обработка ошибок. Каждая служба определяет набор исключений, специфичных для конкретного приложения (например, исключение PaymentRequiredException, исключение NotFoundException, исключение UnauthorizedException), которые перехватываются глобальным плагином страницы состояния Ktor и преобразуются в соответствующие HTTP-ответы со структурированными текстами ошибок. Это гарантирует, что потребители API получают согласованные форматы ошибок, а не необработанные трассировки стека.

Ведение журнала. Ktor's плагин CallLogging от Ktor регистрирует все входящие запросы с указанием их кодов состояния и времени ответа. Этого достаточно для текущего этапа проверки концепции. В производственном развертывании будут добавлены структурированное ведение журнала (например, строки журнала в формате JSON) и агрегирование журналов (например, Datadog, Logtail).

Мобильное приложение объемом 3,6 КМ

Мультиплатформенное мобильное приложение Kotlin предназначено для использования конечными пользователями ProductiveSocial. Он нацелен на Android, iOS и JVM Desktop из единой общей базы кода, используя Compose Multiplatform для уровня пользовательского интерфейса и SQLDelight для локального хранения данных в автономном режиме. Приложение полностью функционально без подключения к сети — все пользовательские взаимодействия сначала записываются в локальную базу данных SQLite, а уровень фоновой синхронизации отправляет изменения в серверные службы и обратно, когда доступно подключение.

3.6.1 Архитектура приложения: трехуровневая чистая архитектура

Приложение состоит из трех отдельных уровней, каждый из которых имеет четко определенные обязанности и не имеет циклических зависимостей.

Уровень домена (commonMain/domain/): Содержит чистые классы данных Kotlin и интерфейсы вариантов использования с нулевой зависимостью от какого-либо фреймворка, базы данных или сетевой библиотеки. Модель предметной области - это простой класс данных Kotlin. Интерфейс GetTasksUseCase определяет, как извлекаются задачи, не зная, поступают ли

они из локальной базы данных или удаленного API. Этот уровень полностью поддается модульному тестированию без какой-либо макетной инфраструктуры.

Уровень данных (commonMain/data/): Содержит реализации репозитория, поддерживаемые двумя источниками данных для каждого типа объекта — LocalDataSource (SQLDelight) и RemoteDataSource (HTTP-клиентKtor). Ответственность репозитория заключается в координации между этими источниками: сначала записи отправляются в local (что делает пользовательский интерфейс отзывчивым немедленно), устанавливается грязный флаг, и диспетчер синхронизации сбрасывает грязные записи на сервер в фоновом режиме. Когда сервер отвечает, локальные строки обновляются идентификаторами, назначенными сервером.

Уровень представления (commonMain/presentation/): Содержит составные элементы пользовательского интерфейса и связанные с ними классы ViewModel. Каждый экран представляет собой составной объект без состояния, который наблюдает за потоком состояний<UIState> из своейViewModel и отправляет пользовательские события по каналу. ViewModels не отображают изменяемое состояние напрямую — все изменения проходят через метод handleEvent(), что делает конечный автомат явным и отслеживаемым.,,,

График зависимостей применяется на границах модуля: домен не имеет зависимостей; данные зависят от домена; представление зависит от данных и домена. Наборы исходных текстов для конкретной платформы (androidMain, iosMain, jvmMain) содержат только регистрации драйверов для конкретной платформы и точки входа для внедрения зависимостей - вся бизнес-логика находится в commonMain.

3.6.2 Локальная база данных: SQLDelight схема

У каждого объекта есть локальная таблица SQLDelight. Схема предназначена для зеркального отображения серверной модели данных при добавлении полей, необходимых для автономных операций.

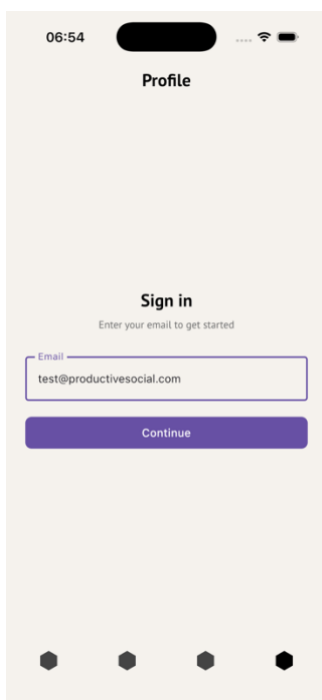
Table	Purpose
<code>task</code>	Stores tasks with priority, urgency, due date, recurrency, completion state
<code>habit</code>	Stores habits with type (START/QUIT), recurrency, scheduled times
<code>routine</code>	Stores routines with ordered steps
<code>routine_step</code>	Individual steps of a routine with duration and autoStart flag
<code>habit_completion</code>	One row per day a habit was marked complete
<code>routine_completion</code>	One row per routine completion with step-level detail
<code>completion_log</code>	General completion events for tasks
<code>tag</code>	User-scoped string labels shared across entity types
<code>project</code>	Project containers with name, color, icon, priority
<code>pomodoro_settings</code>	User's timer configuration
<code>pomodoro_session</code>	One row per focus session with entity link and cumulative stats
<code>pomodoro_interval</code>	One row per work/break interval within a session
<code>sync_metadata</code>	Key-value store: deviceId, serverUserId, lastSyncedAt, lastTimerSyncedAt, stored JWT

3.6.2 Аутентификация

Используйте для входа, используя только их адрес электронной почты — без пароля. На экране аутентификации отображается один ввод электронной почты и кнопка “Продолжить”. При отправке приложение вызывает `POST /api/v1/auth/identify` в `selfmanager`. Сервер находит или создает пользователя и возвращает токен доступа, токен обновления и целое число `userId`.

Токен и `userId` записываются в таблицу `sync_metadata`. При последующих запусках приложения `UserSessionManager.init()` считывает сохраненный токен и автоматически восстанавливает сеанс — пользователь попадает непосредственно на главный экран без повторной аутентификации. Сеанс остается действительным до истечения срока действия токена (24 часа с момента выдачи) или до явного выхода пользователя из системы.

Выход из системы вызывает `UserSessionManager.logout()`, который стирает все локально сохраненные пользовательские данные — токен, идентификатор пользователя, все таблицы сущностей и все состояние синхронизации — эффективно возвращая приложение в чистое состояние при первом запуске.



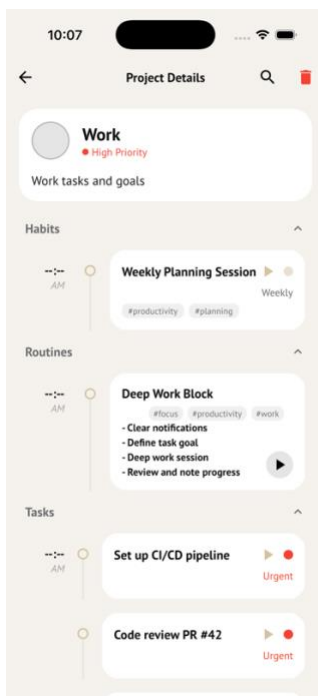
```
Client → POST /api/v1/auth/identify { "email": "user@example.com" }  
← { accessToken, refreshToken, userId, expiresIn }
```

3.6.5 Проекты

Проекты - это организационные контейнеры. Каждая сущность в self-manager принадлежит проекту. У каждого проекта есть имя, описание, шестнадцатеричный цветовой код, название значка, уровень приоритета и теги. Пользователь может просматривать все проекты в виде таблицы, каждый из которых отображается в виде цветной карточки с названием проекта и количеством объектов.

При детализации проекта открывается подробное представление проекта, в котором отображаются все вложенные в него задачи, привычки и подпрограммы, а также теги и приоритет проекта. Проекты могут создаваться локально (сразу видны в таблице) и синхронизироваться с сервером в фоновом режиме.

При первой синхронизации у каждого пользователя есть по крайней мере проект “По умолчанию”, созданный автоматически selfmanager. Локальная база данных хранит как клиентский идентификатор синхронизации UUID, так и назначенное сервером целое число ServerID; после первого ответа синхронизации ServerID заполняется и используется для всех последующих межведомственных ссылок.



3.6.6 Экран самостоятельного управления

Экран самостоятельного управления - это центральное окно ежедневного планирования и главный экран приложения. Он объединяет задачи, привычки и распорядок дня в единый унифицированный список на выбранную дату.

В верхней части экрана находится **горизонтальная полоса даты** — прокручиваемый ряд фишек с датами, показывающий текущую неделю. Нажатие на чип даты обновляет выбранную дату и перезагружает список. Пользователь может проводить пальцем влево и вправо для перехода между неделями. Кнопка “Сегодня” возвращает полосу даты к текущей дате.

Под полосой даты находится **единый список производительности**, который показывает все задачи, запланированные на выбранную дату, все привычки, которые должны быть выполнены в этот день (в зависимости от их повторяемости), и все процедуры, запланированные на этот день, — все в одном прокручиваемом списке. Каждый элемент можно отличить по значку: флажок для задач, значок flame для запуска привычек, значок X для прекращения привычек и значок контрольного списка для процедур.

Список поддерживает два ортогональных элемента управления в заголовке:

Сортировка: товары могут быть отсортированы по срокам выполнения (по возрастанию или по убыванию) или по приоритету (от высокого к низкому).

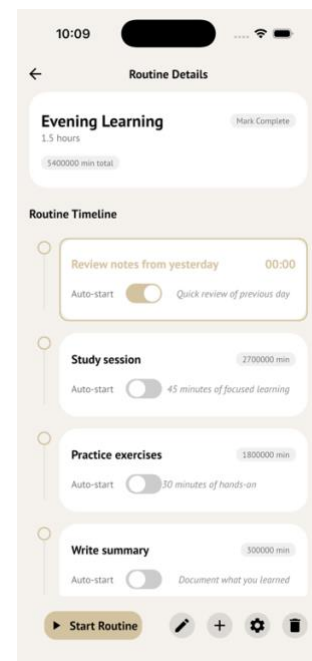
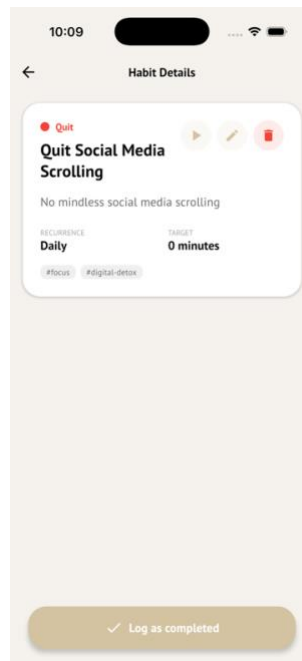
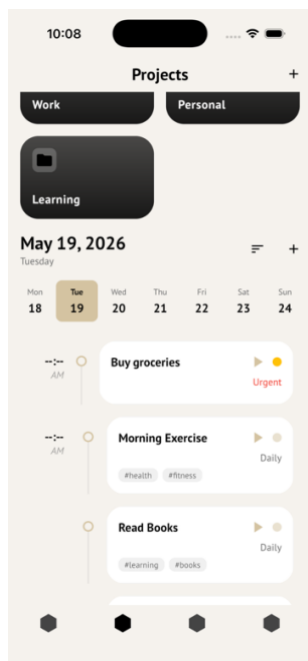
Группировка: элементы могут быть сгруппированы по отсутствию (плоский список), по тегу, по проекту или по срочности. При группировании каждая

группа представляет собой складной заголовок раздела с названием группы и количеством элементов.

The *SelfManagerViewModel* объединяет реактивные потоки из репозитория задач, привычек и рутин с помощью оператора объединения сопрограмм Kotlin, применяет текущую конфигурацию сортировки и группировки и повторно отправляет обработанный список всякий раз, когда изменяется выбранная дата или какие-либо данные репозитория. Это означает, что если пользователь отмечает привычку завершенной в этом списке, список обновляется мгновенно без перезагрузки.

В этом списке каждый элемент доступен для встроенного действия:

- Задачи: нажатие флажка отмечает выполнение задачи; длительное нажатие открывает контекстное меню (редактировать, удалить, начать сеанс фокусировки)
- Привычки: нажатие на кружок завершения отмечает привычку, выполненную на сегодня; если у привычки есть подзадачи, появится нижний лист для пошаговой разметки
- Процедуры: нажатие на процедуру открывает программу выполнения процедуры.



3.6.7 Задачи

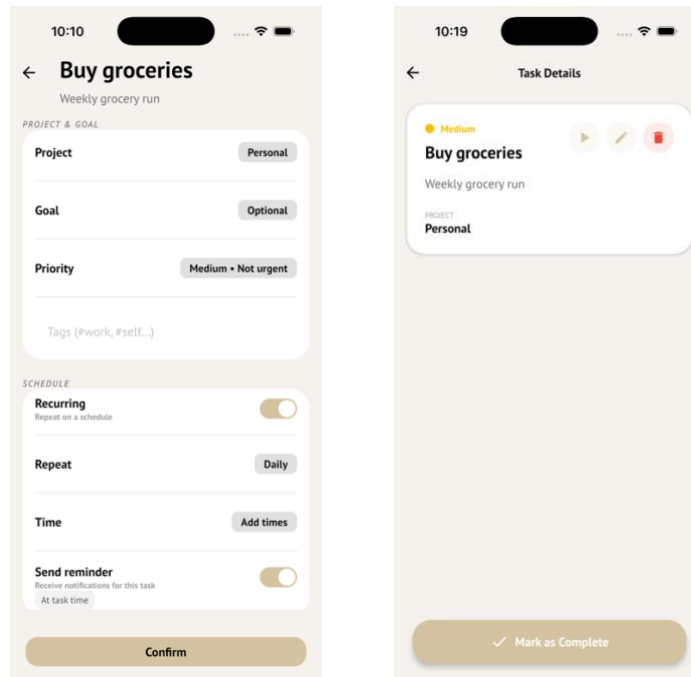
Задачи являются основной единицей целенаправленной работы. Полная модель данных о задачах включает:

- **Имя** и необязательное **описание**
- **Приоритет**: Отсутствует, Низкий, средний или Высокий — отображается в виде значка с цветовой кодировкой
- **Срочность**: измерение, отдельное от приоритета (допускающее матрицу срочности / приоритета 2×2)
- **Дата выполнения**: необязательно, с выделением просрочки по истечении
- **Повторяющийся флаг**: помечает задачи, которые повторяются по расписанию
- **Цель**: необязательная количественная цель (например, “прочитать 30 страниц”)
- **Запланированное время**: одно или несколько конкретных периодов дня, когда планируется выполнение задачи, сохраненных в виде строк ЧЧ: мм
- **Параметры напоминания**: срабатывание уведомления в запланированное время или за 5, 10, 15 или 30 минут до этого
- **Теги**: несколько меток строк, общих для всех типов сущностей
- **Подзадачи**: упорядоченный список подпунктов, у каждого из которых есть имя и состояние завершения; подзадачи могут быть переупорядочены, и у каждой есть собственное отслеживание выполнения
- **Время, затраченное**: накопленные минуты связанных сеансов Pomodoro, автоматически обновляемые службой таймера через внутреннюю конечную точку регистрации времени
- **Журнал выполнения**: записи с отметками времени о каждом времени, когда задача была отмечена как выполненная (для повторяющихся задач отслеживается история повторений)

Экран **создания и редактирования задач** управляет всем вышеперечисленным в единой прокручиваемой форме. Подзадачи могут быть добавлены встроенными с помощью текстового поля в нижней части списка подзадач. Теги вводятся как чипы с автозаполнением в соответствии с существующей библиотекой тегов пользователя.

Представление **сведений о задаче** - это аналог редактора, ориентированный на чтение. В нем представлены полные данные задачи, контрольный список подзадач, накопленный счетчик затраченного времени, связанные сеансы

Pomodoro с их датами и продолжительностью, а также временная шкала журнала выполнения.



3.6.8 Привычки

Привычки бывают двух типов, отражающих противоположные поведенческие цели:

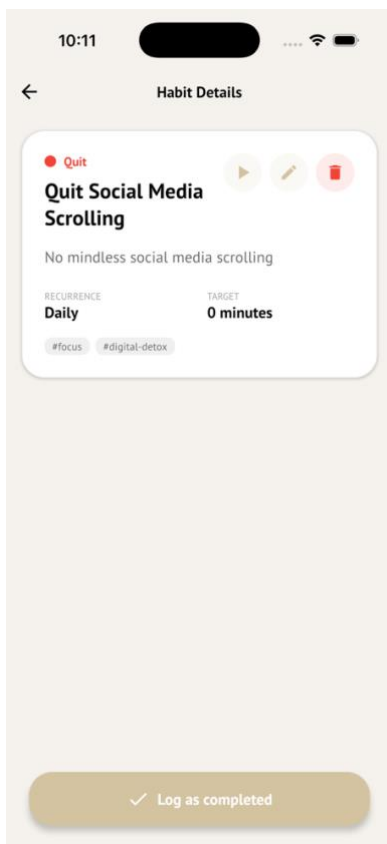
Начальные привычки представляют собой модели поведения, которые пользователь хочет выработать — ежедневные упражнения, медитация, чтение в течение 30 минут. Пользовательский интерфейс завершения разработан с учетом поощрения: анимация круговой заливки при пометке "Завершено", счетчик растущих полос и календарь тепловой карты, показывающий последние 30 дней завершения.

Привычки к отказу от курения представляют собой поведение, от которого пользователь хочет избавиться — первым делом проверять социальные сети по утрам, есть сахар после 8 вечера. Пользовательский интерфейс завершения инвертирует положительное обрамление: завершение означает, что пользователь успешно избежал такого поведения. Применяются те же правила серии и календаря, но язык меняется.

Оба типа используют одну и ту же модель данных: название, описание, проект, приоритет, теги, повторяемость (ежедневно, еженедельно и т.д.), запланированное время (когда планируется выполнение привычки), время напоминания (триггеры уведомлений) и подзадачи (отдельные шаги в рамках привычки, каждый из которых может быть выполнен независимо).

Выполнение привычки отслеживается с ежедневной детализацией в таблице `habit_completion`: в каждой строке записывается идентификатор привычки, дата завершения и, при необходимости, состояния завершения отдельных подзадач. Датой завершения является местная дата (не временная метка UTC), чтобы избежать прерывания полосы, вызванного часовым поясом, когда пользователь отмечает завершение привычки ближе к полуночи.

Пометка привычки завершенной немедленно создает локальную строку `habit_completion`, добавляет ее в очередь синхронизации и обновляет счетчик серий в пользовательском интерфейсе. Если в привычке есть подзадачи, в нижней таблице завершения каждая подзадача представлена в виде флажка и записей, которые были выполнены в какое время.



3.6.9 Процедуры

Процедуры представляют собой структурированную последовательность шагов — упорядоченный контрольный список, в котором шаги должны выполняться по порядку, а не в произвольной последовательности. Они предназначены для фиксированных рабочих процессов, которые пользователь выполняет неоднократно: утренняя рутина, последовательность действий перед тренировкой, протокол учебного занятия, подведение итогов в конце дня.

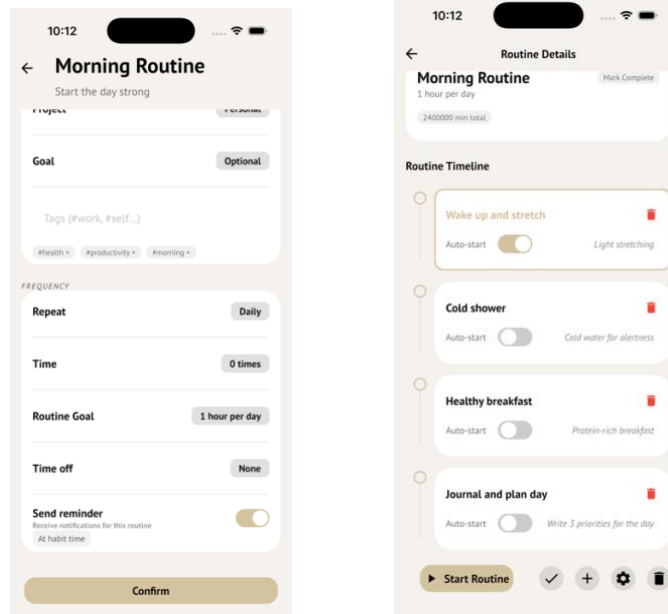
Каждая процедура имеет те же данные заголовка, что и задачи и привычки (название, описание, проект, приоритет, теги, повторяемость, запланированное время, время напоминания). Отличительной особенностью является его список шагов: упорядоченный набор шагов, каждый из которых содержит:

- **Имя:** что делать на этом этапе
- **Продолжительность** в минутах: сколько времени обычно занимает выполнение шага
- **Индекспозиции:** определяет фиксированный порядок
- **Флаг автозапуска:** должно ли приложение автоматически переходить к этому шагу по завершении предыдущего

Процедуры также поддерживают нерабочее время — определенные даты, дни недели или временные интервалы, когда процедура не должна отображаться в списке самостоятельного управления. Это позволяет пользователю планировать выполнение программы только в будние дни или приостанавливать ее в праздничные дни.

The Routine Runner - это специальный интерфейс выполнения, который открывается, когда пользователь выбирает процедуру в списке самостоятельного управления. В нем представлен шаг за шагом:

7. Название этапа и планируемая продолжительность указаны вверху
8. Таймер обратного отсчета запускается в течение запланированного времени (если значение Автозапуск равно true, он запускается автоматически; в противном случае пользователь нажимает “Start Step”)
9. Пользователь нажимает “Готово”, чтобы отметить шаг завершенным и перейти к следующему, или “Пропустить”, чтобы двигаться дальше, не отмечая его завершенным
10. Индикатор выполнения вверху показывает, сколько шагов было выполнено из общего числа
11. Когда выполнен последний шаг, сводка завершения показывает общее затраченное время и выполненные / пропущенные шаги, а запись routine_completion записывается в локальную базу данных с подробностями на уровне шага



3.6.10 Таймер для приготовления Помидоров

Модуль Pomodoro является наиболее интерактивным компонентом приложения, реализующим настраиваемый таймер фокусировки с полным жизненным циклом сеанса, отслеживаемым локально и синхронизируемым со службой таймера.

Настройки управляют поведением таймера и хранятся в локальной таблице `pomodoro_settings`. Настраиваемыми параметрами являются:

- Продолжительность рабочего интервала (по умолчанию 25 минут)
- Продолжительность короткого перерыва (по умолчанию 5 минут)
- Длительный перерыв (по умолчанию 15 минут)
- Количество циклов перед длительным перерывом (по умолчанию 4)
- Автоматический запуск перерывов: запускается ли таймер перерыва автоматически по окончании рабочего интервала
- Автоматический запуск следующего сеанса: запускается ли рабочий таймер автоматически после перерыва
- Звук: воспроизводятся ли звуковые сигналы на границах интервала
- Уведомления: срабатывают ли системные уведомления по окончании интервалов

Изменения настроек записываются локально и синхронизируются со службой таймера через `PUT /api/v1/pomodoro/settings`. Когда ответ настроек включает рекомендуемый блок (доступный после того, как пользователь накопит достаточно истории сеансов), на экране настроек отображаются рассчитанные

сервером персонализированные длительности наряду с текущими настройками в качестве сравнения.

Создание сеанса и привязка объекта. Сеанс Pomodoro может быть создан в двух режимах:

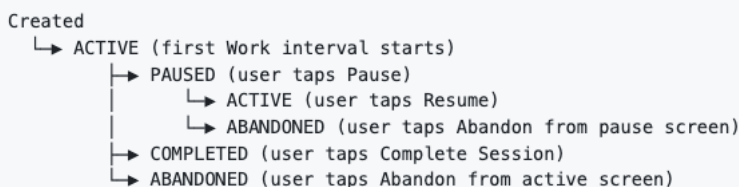
Автономный режим: Пользователь открывает таймер и нажимает “Начать сеанс фокусировки” без привязки к какому-либо объекту. Сеанс создается локально, начинается первый рабочий интервал и начинается обратный отсчет. Этот режим предназначен для общего времени фокусировки, не привязанного к конкретной задаче.

Связанный режим: Пользователь нажимает на значок помидора рядом с задачей, привычкой или рутинной. FocusViewModel открывает таймер, предварительно связанный с этим объектом. При этом записывается тип объекта и локальный идентификатор в строке сеанса (EntityType и entityLocalId). Когда сеанс завершается и синхронизируется с сервером, служба таймера запускает вызов журнала времени для selfmanager, который добавляет общее количество рабочих минут сеанса к time_spent_minutes связанного объекта. Это автоматическое определение времени не требует ручного ввода от пользователя.

The PomoViewModel владеет текущим состоянием таймера: временем, оставшимся в текущем интервале, типом текущего интервала (работа / Короткий перерыв / Длительный перерыв), текущим количеством циклов, активен ли перерыв, общим количеством завершенных сегодня сеансов и общим количеством потраченных минут на сегодняшний день. Когда интервал заканчивается, ViewModel вычисляет следующий тип интервала на основе количества циклов и настроек, затем автоматически запускает его, если установлен флаг автозапуска.

The FocusViewModel расширяет PomoViewModel для сеансов, связанных с объектами. Это дополнительно загружает подзадачи связанного объекта, которые отображаются на складной панели под таймером во время рабочего интервала. Пользователь может отмечать подзадачи во время сеанса, не покидая экран таймера. После завершения сеанса FocusViewModel отображает средство выбора задач, чтобы пользователь мог немедленно перейти к следующей задаче фокусировки, не возвращаясь к самостоятельному менеджеру.

Состояния жизненного цикла сеанса и их переходы:



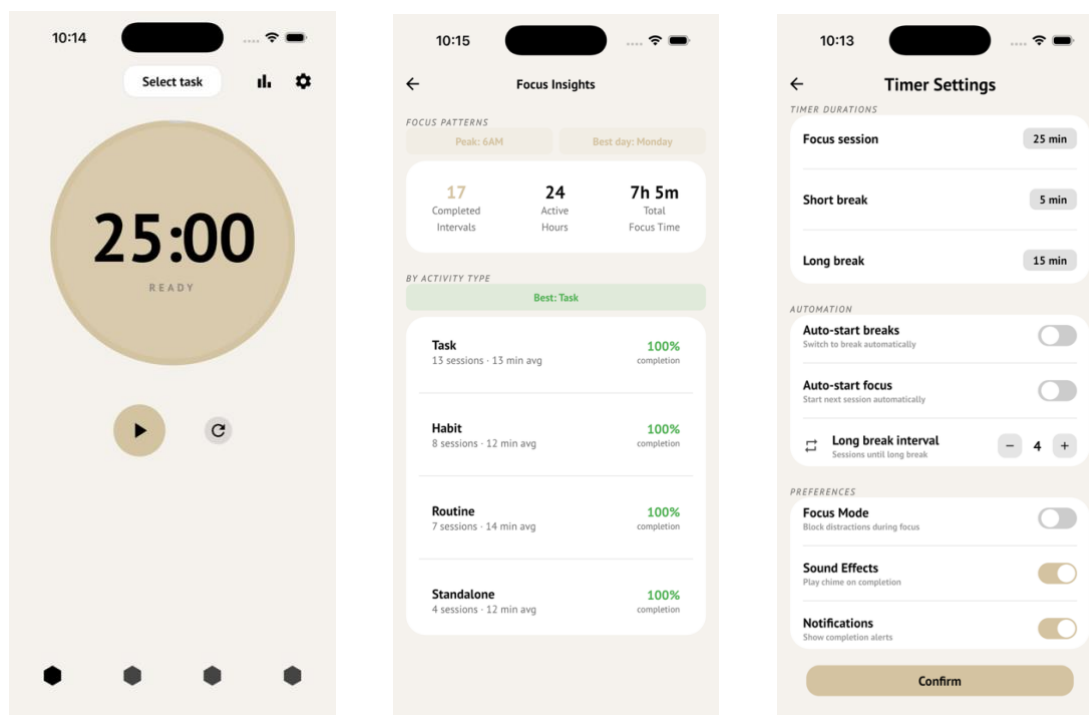
Каждый переход состояния обновляет локальную строку `pomodoro_session` и помечает ее как "грязную". Строка передается службе таймера при следующей синхронизации через `POST /api/v1 /pomodoro/sync`, который следует тому же пакетному идиоматичному шаблону, что `isync otselfmanager`.

Сигнал о риске оставления. Когда пользователь нажимает "Пауза", `ViewModel` проверяет, достигло ли количество пауз персонального порога пользователя. Пороговое значение вычисляется на основе истории их отказа (среднее количество пауз при отказе, минимум 3, если истории недостаточно). Если пороговое значение достигнуто, на экране паузы отображается предупреждение: "Вы делали паузу столько раз, прежде чем отказаться. Остаться в этом?" с возможностью возобновить или Отказаться в качестве двух вариантов.

Отклоняйте предложения. После каждого завершеного рабочего интервала таймер вычисляет, рекомендуется ли стандартный короткий перерыв, длительный перерыв или продолжительный отдых, исходя из общего времени сосредоточения пользователя сегодня и того, был ли уже сделан длительный перерыв. Предложение о перерыве отображается в виде короткого сообщения с рекомендуемой продолжительностью. Пользователь может принять предложение или вручную выбрать другой тип перерыва.

Экран статистики Pomodoro. Отдельный экран статистики (доступный на вкладке Pomodoro) показывает:

- Общее количество сеансов и общее количество целенаправленных минут (за все время и на этой неделе)
- Коэффициент завершения сеанса (завершенный по сравнению с брошенным)
- Шаблоны фокусировки: столбчатая диаграмма, показывающая интервалы выполненной работы по часам суток и дням недели, определяющая время максимальной фокусировки
- Статистика сущности: разбивка сеансов и рабочих минут по типам сущности (задачи, привычки, процедуры, автономные)



3.6.11 Журнал

Модуль журнала является одной из наиболее стратегически важных запланированных функций платформы. Журнал служит отражающему аспекту личной продуктивности. В отличие от задач (ориентированных на завершение) и привычек (ориентированных на последовательность), ведение дневника создает пространство для ретроспективного осмысления: объединения событий дня в связное повествование, осмысления того, как прошел день, и извлечения уроков на завтра.

В каждой записи журнала будет записано:

- Дата и время написания
- Индикатор настроения: простая категориальная оценка настроения на основе шкалы (например, 1-5 или на основе эмодзи), полученная на момент написания
- Контент в свободной форме: собственный текст пользователя
- Контекст автоматической активности: структурированное резюме выполненных задач за день, выполнение привычек и общее количество сеансов Pomodoro — автоматически прикрепляется к записи из других сервисов, поэтому пользователю не нужно вручную расшифровывать то, что он сделал

Ключевым отличием является последний момент. Запись в дневнике в Первый день или любом другом приложении для ведения журнала требует, чтобы пользователь запомнил и вручную записал, чего он достиг. В ProductiveSocial запись в журнале предварительно заполняется структурированным резюме основной деятельности за день - каждой выполненной задачи, каждой отмеченной сделанной привычки, общего времени сосредоточения, которое автоматически определяется с помощью selfmanager и службы таймера. Записи пользователей заполняют субъективный слой: как они себя чувствовали, что было трудно, что их удивило.

Синтез с помощью искусственного интеллекта. Самой мощной функцией журнала, запланированной как часть сервиса `psocial_journal`, является режим синтеза искусственного интеллекта. Когда пользователь завершает запись в журнале, специальный тип анализа (`DAILY_REFLECTION`) в службе аналитики будет объединять:

- Письменный текст пользователя (субъективный отчет)
- Выполнение задач на день и приоритеты (от `selfmanager`)
- Запись о завершении привычки (из `selfmanager`)
- Данные сеанса Pomodoro, включая время фокусировки и атрибуцию объекта (по таймеру)

Магистр права обобщит это в краткую, персонализированную информацию о конце рабочего дня, определяя закономерности (“Вы выполнили 4 из 5 своих высокоприоритетных задач, и ваше время сосредоточения было выше среднего за неделю, но вы пропустили вечернюю рутину в третий раз на этой неделе”), выявляя корреляции и предлагая одно конкретное намерение на завтра.

Этот синтез максимально приближает платформу к видению, описанному в заявлении о миссии: преобразование исходных данных о деятельности и субъективного опыта в действенный рассказ. Ни одно существующее приложение для повышения производительности или ведения журнала в настоящее время не выполняет этого, поскольку ни одно существующее приложение не имеет одновременного доступа ко всем этим источникам данных.

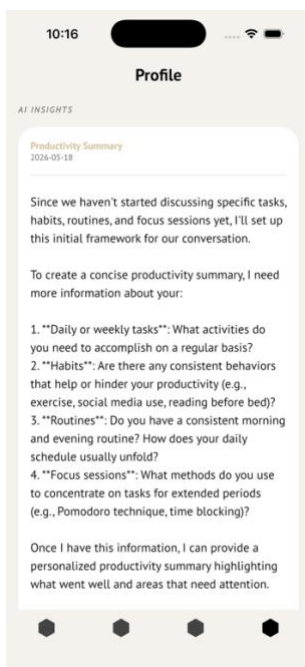
Планируемая архитектура обслуживания журналов. Сервис `psocial_journal` будет следовать тому же архитектурному шаблону, что и другие сервисы: Kotlin / Ktor, собственная база данных PostgreSQL, аутентификация JWT с использованием общего секрета и внутренние конечные точки для службы аналитики для извлечения содержимого журнала при создании подсказок искусственного интеллекта. Записи журнала будут синхронизированы с

мобильным клиентом по тому же протоколу пакетной синхронизации, который используется selfmanager.

3.6.12 Аналитические данные (страница пользователя)

Экран страницы пользователя интегрирует функции анализа с помощью искусственного интеллекта сервиса analytics непосредственно в мобильное приложение. AnalyticsApiClient (в commonMain) вызывает POST /api/v1/analytics с типом анализа и необязательным идентификатором модели. Поддерживаемые типы анализа: PRODUCTIVITY_SUMMARY, TASK_PRIORITIZATION, HABIT_INSIGHTS, ROUTINE_OPTIMIZATION, WEEKLY_TIMER_SUMMARY и ПОЛЬЗОВАТЕЛЬСКИЙ.

The InsightsViewModel управляет потоком запуска анализа: он вызывает API, обрабатывает состояние загрузки (показывает индикатор выполнения с пояснительным сообщением, поскольку вывод может занимать 15-30 секунд) и сохраняет результат в потоке состояний<InsightUiState>, который отображается на экране. Прошлые отчеты извлекаются из GET /api/v1 / analytics и отображаются в виде прокручиваемого списка истории. Текущий кредитный баланс отображается в верхней части экрана и выбирается из GET /api/v1/credits/balance.



3.6.13 Адаптивный макет

Приложение адаптируется к размеру экрана с помощью утилиты windowSize, которая классифицирует текущий экран как компактный, средний или расширенный.

На компактных экранах (телефонах) навигация представляет собой нижнюю панель навигации — стандартный шаблон мобильной навигации, видимый в нижней части экрана со значками + ярлыками для четырех пунктов назначения верхнего уровня.

На экранах среднего размера и увеличенного размера (планшеты и настольные компьютеры) навигация переключается на боковую панель навигации — вертикальный столбец значков и меток вдоль левого края, освобождающий для содержимого всю ширину экрана. На больших экранах рабочего стола отображается полный навигационный блок с расширенными надписями.

С помощью составного элемента `NavScaffold` создается соответствующий навигационный компонент с областью содержимого на основе текущего размера `windowSize`, в котором находится окно. Все составные файлы адаптируют свой макет к доступной ширине экранов — списки используют многоколоночные сетки на широких экранах и одностолбцовые списки на телефонах.

3.6.14 Протокол автономной синхронизации

Уровень синхронизации является основой архитектуры, ориентированной на автономный режим. Каждому объекту присваивается сгенерированный клиентом `UUID` (`syncId`) во время создания. Этот `UUID` является ключом идемпотентности — если один и тот же объект отправляется дважды (из-за повторной попытки), сервер распознает дубликат `syncId` и возвращает существующий идентификатор сервера вместо создания второй записи.

Selfmanager sync - это одиночный двунаправленный вызов: `POST /api/v1/sync`. Текст запроса содержит:

- создает: все объекты, созданные локально с момента последней синхронизации (с идентификатором синхронизации и полными данными поля)
- обновления: все объекты, измененные локально с момента последней синхронизации (с идентификатором синхронизации и измененными полями)
- удаляет: синхронизирует идентификаторы локально удаленных объектов (надгробий)
- `lastSyncedAt`: временная метка, сохраненная из предыдущего ответа синхронизации

Сервер обрабатывает их в порядке зависимости (проекты перед задачами, привычки перед завершениями), применяет семантику `upsert` для создания и обновлений и возвращает:

- idMappings: { clientSyncId → serverIntegerId } для всех обрабатываемых объектов, чтобы локальная база данных могла заполнять свои ServerID столбцы
- Изменения сервера: полные объекты сущностей для всех записей, измененных на сервере с момента последней синхронизации (например, изменения, внесенные с веб-панели мониторинга или другого устройства)
- ошибки: сообщения об ошибках для каждого элемента для любых записей, которые не удалось обработать
- Новая временная метка syncedAt для следующего вызова

Получив ответ, диспетчер синхронизации:

1. Обновляет все локальные строки, которые имеют IsDirty = true, с их идентификаторами сервера и устанавливает IsDirty = false
2. Удаляет все локальные строки, помеченные как IsDeleted = true (надгробия подтверждены сервером)
3. Обновляет все серверные изменения в локальной базе данных (сервер выигрывает при любых конфликтах)
4. Сохраняет новую временную метку syncedAt в sync_metadata

Синхронизация по таймеру выполняется по тому же шаблону через POST /api/v1/pomodoro/sync в службе таймера, используя lastTimerSyncedAt из таблицы sync_metadata.....

Когда подключение недоступно, все записи отправляются в local SQLDelight с IsDirty = true. Диспетчер синхронизации повторяет попытки с настраиваемым интервалом времени при восстановлении подключения. Поскольку все операции синхронизации являются идемпотентными, отсутствует риск дублирования данных при повторных попытках.

3.6 Текущее состояние реализации

Module	Status
Project structure and build (Android, iOS, Desktop)	Complete
SQLDelight schema (all entity tables + sync_metadata)	Complete
Clean architecture layers (domain, data, presentation interfaces)	Complete
Koin dependency injection (all modules)	Complete
Authentication screen	Complete and functional
API clients (selfmanager, timer, analytics)	Scaffolded
Sync manager	Scaffolded
Projects screen	In progress
Self-Manager screen (tasks + habits + routines, date strip, sort/group)	In progress
Tasks CRUD and detail	In progress
Habits CRUD, completion, and detail	In progress
Routines CRUD, Routine Runner	In progress
Pomodoro timer (PomoViewModel, FocusViewModel)	In progress
Pomodoro statistics	Planned
Analytics / Insights (User Page)	Scaffolded
Journal	Placeholder screen
Credits display	Planned
Responsive layout (NavScaffold)	Complete

3.7 Определение области применения MVP

MVP ProductiveSocial был определен как минимальный набор возможностей, который демонстрирует основное ценностное предложение — унифицированное отслеживание производительности с междоменным анализом ИИ - способом, доступным реальному пользователю.

MVP включает в себя четыре серверных сервиса, две панели мониторинга и API surface, необходимые для поддержки мобильного приложения. Он явно не включает в себя: модуль журнала, социальные функции, проприетарные

обученные модели, хостинг LLM производственного уровня, а также полнофункциональные серверные сервисы и мобильное приложение.

Эта область была выбрана по трем причинам:

12. Основное ценностное предложение (унифицированный междоменный анализ искусственного интеллекта) полностью демонстрируется при текущей реализации. Пользователь может добавлять свои задачи, привычки, распорядок дня и сеансы фокусировки и получать качественно более подробный анализ, чем может предоставить любое существующее приложение.
13. Отложенные функции (журнал, социальные сети) требуют значительных проектных работ, выходящих за рамки серверной архитектуры — они связаны со сложными задачами пользовательского интерфейса, которые лучше всего решать после проверки основных модулей повышения производительности.
14. Архитектура явно учитывает отложенные функции. В модели данных предусмотрены ячейки для записей журнала и заметок (объектов, которые ссылаются на ту же личность пользователя и структуру проекта, что и задачи и привычки). Социальная функция может быть реализована как новый микросервис без изменения какого-либо существующего сервиса.

3.8 Планируемые сервисы и функции

3.8.1 Расширенный самоменеджмент: помимо задач, привычек и рутины

Текущий сервис selfmanager содержит три структурированных типа объектов производительности — задачи, привычки и рутины. Это объекты высочайшей сложности и ценности в области личной продуктивности, которые были правильной отправной точкой. Однако полнофункциональное приложение для управления задачами и персональной организации включает в себя более широкий спектр типов сущностей, которые служат для различных вариантов использования.

Списки дел и быстрый просмотр входящих сообщений. Не каждый элемент, который пользователь хочет зафиксировать, представляет собой структурированную задачу с приоритетом, сроком выполнения и подзадачами. Многие потребности в производительности проще: список покупок, список рекомендаций по книгам, небольшая заметка для себя. Списки дел - это легковесные, неупорядоченные или произвольно упорядоченные списки однострочных элементов, без механизма планирования и приоритизации полной задачи. "Входящие с быстрым захватом" - это специальный проект, который получает все элементы, которые пользователь фиксирует, без их немедленной классификации — временная промежуточная площадка для мыслей и обязательств, которые будут обработаны (перемещены в нужный проект и тип объекта) во время еженедельного обзора.

Контрольные списки. Контрольные списки - это списки элементов многократного использования, которые не являются повторяющимися привычками или рутинами, но используются неоднократно в похожих контекстах. Предполетный чек-лист для рабочей поездки, список вещей, которые нужно проверить перед отправкой отчета, чек-лист после тренировки — это не привычки (они не отслеживаются на предмет ежедневных нагрузок) и не рутины (они не запланированы), но они выигрывают от структуры упорядоченного списка с отслеживанием выполнения.

Шаблоны. Шаблоны проектов - это предопределенные структуры, которые пользователь может создавать при запуске новой работы. Шаблон “Начало нового проекта” может предварительно заполнять набор задач (определить область применения, запланировать стартовую встречу, определить заинтересованных лиц), которые обычно требуются пользователю для каждого нового проекта. Шаблоны уменьшают трудности при запуске повторяющихся типов проектов.

Просмотр календаря. Навигация по полоске даты на экране самостоятельного менеджера - это современный подход к организации. Полный просмотр

календаря — ежемесячные и еженедельные форматы таблиц, показывающие запланированные задачи, сроки выполнения привычек и рутинные графики на расширенном временном горизонте — дает пользователю более широкую временную перспективу своих обязательств и помогает определить дни с превышением графика до их наступления.

Заметки со связыванием сущностей. Заметки - это текстовые документы произвольной формы, которые используются в контексте производительности. Отличительная черта заметок в ProductiveSocial (по сравнению отдельное приложение для заметок) - это привязка объекта: заметка может быть прикреплена к проекту, задаче, привычке или рутине. Заметки о собрании прикрепляются к задаче “Посетить совещание по планированию третьего квартала”. Исследовательские заметки прилагаются к проекту, о котором они информируют. Это контекстное вложение означает, что заметки всегда можно найти через объект, к которому они относятся, а не только через интерфейс поиска.

3.8.2 Расширенный таймер: общий фокус и полосы

Текущий Pomodoro timer - это инструмент персональной ориентации. Планируемые расширения привносят социальные аспекты и геймификацию в работу с focus.

Полосы фокусировки. Полоса фокусировки отсчитывает дни подряд, в течение которых пользователь завершил хотя бы один сеанс Pomodoro. Подобно полосам привычек, полосы сосредоточенности используют неприятие потерь для поощрения постоянной ежедневной вовлеченности. Счетчик серий будет отображаться на видном месте на главном экране таймера и будет виден в общедоступном профиле пользователя.

Цели сеанса. Перед началом сеанса пользователь может установить цель сеанса — краткое текстовое описание того, чего он намерен достичь в этом блоке. По завершении сеанса цель представляется вместе с кратким описанием сеанса, что позволяет пользователю поразмыслить о том, достигли ли они того, что намеревались сделать. Цели сеанса сохраняются в записи сеанса и становятся частью данных, доступных службе аналитики при создании анализа с помощью искусственного интеллекта.

Общие комнаты фокусировки. Комната фокусировки - это виртуальное общее пространство, где несколько пользователей одновременно выполняют фокусировку. Таймер каждого участника работает независимо (они могут иметь разную продолжительность и схему перерывов), но комната показывает состояние всех участников в режиме реального времени (сосредоточены / на перерыве / завершены). Социальная заметность создает мягкий эффект

подотчетности — знание того, что другие могут видеть, сосредоточены ли вы или отказались от сеанса, увеличивает цену отказа. Фокус-комнаты будут реализованы как часть социальной службы.

3.8.3 Социальный слой

Социальный модуль - это функция, которая наиболее четко отличает ProductiveSocial от инструмента личной продуктивности и превращает его в платформу. В основе его дизайна лежит один принцип: использовать мотивирующую силу социальной ответственности, не привлекая внимания основных социальных сетей.

Проблема фрагментации социальных сетей. Основные социальные платформы (Instagram, Twitter / X, TikTok) предназначены для максимального увеличения времени пребывания на платформе за счет бесконечной прокрутки, алгоритмического добавления ленты и повторного вовлечения с помощью уведомлений. Эти механизмы прямо противоречат цели целенаправленной продуктивной работы. Внедрение обычной социальной ленты в приложение для повышения производительности подорвет саму цель приложения.

The Продуктивно-социальный социальный дизайн. Социальный модуль архитектурно отделен от рабочего пространства для повышения производительности. На экранах самообслуживания или таймера нет встроенной социальной ленты. Социальные функции доступны через специальную страницу и предназначены для обмена выполненной работой и постоянными привычками, а не эфемерным контентом.

Планируемыми социальными функциями являются:

Общедоступные профили. У каждого пользователя есть общедоступная страница профиля с отображаемым именем, биографией и общедоступными достижениями. Профиль не отображает статус в режиме реального времени или прямую трансляцию активности — это сводка достижений, а не интерфейс наблюдения.

Обмен достижениями. Платформа определит набор достижений, заработанных за достижение контрольных показателей: выполнение определенного количества задач, поддержание привычки в течение 30 дней, накопление 100 часов сосредоточенности, выполнение полной недели утренних процедур. Заработанные достижения отображаются в виде значков в общедоступном профиле пользователя и могут быть опубликованы в ленте сообщества.

Схемы привычек. Пользователи, выработавшие успешные привычки, могут публиковать их в виде схем - общих шаблонов, которые могут перенять другие пользователи. Схема привычек “Ежедневное чтение продолжительностью 30

минут”, опубликованная одним пользователем, может быть принята другим пользователем одним нажатием, предварительно заполнив всю конфигурацию привычек (тип, повторяемость, запланированное время, напоминания). Это создает одноранговую базу знаний об эффективных привычках.

Лента сообщества. Простая хронологическая лента достижений, которыми делятся пользователи, на которые подписан текущий пользователь. В отличие от алгоритмических социальных каналов, в ленте сообщества нет рейтинга, продвигаемого контента и уведомлений. Это низкая осведомленность окружающих о том, чего добиваются другие в вашей сети, предназначенная для просмотра один раз в день, а не для принудительной проверки.

Партнеры по подотчетности. Пользователь может назначить конкретные связи партнерами по подотчетности для достижения определенной привычки или цели. Партнеры по подотчетности получают еженедельный отчет о прогрессе друг друга в достижении общей цели — тихий, не вызывающий трения толчок, который имеет более сильную эмпирическую поддержку для изменения поведения, чем общественное давление.

Архитектура. Социальный сервис будет представлять собой новый набор независимых микросервисов, следующих тем же шаблонам, что и существующие сервисы: Kotlin / Ktor, собственная база данных PostgreSQL, аутентификация JWT с помощью общего секрета и внутренние конечные точки для аналитического сервиса для включения социальных данных (например, показателей подотчетности партнеров) в анализ искусственного интеллекта. Он будет использовать данные из selfmanager и таймера через внутренние API, но не будет владеть копией данных о производительности — он будет ссылаться на идентификаторы объектов и идентификаторы пользователей, не дублируя сами объекты.

3.8.4 Пользовательский Сервис

В настоящее время идентификация пользователя в ProductiveSocial минимальна: адрес электронной почты, назначенный сервером целочисленный идентификатор и JWT. Этого достаточно для текущего этапа, когда платформа не имеет социального измерения и не ориентирована на общественность.

При внедрении социальных функций требуется более полное представление о пользователе: отображаемое имя, фотография профиля, биография, часовой пояс, настройки уведомлений, настройки конфиденциальности и социальный график (кто за кем следит). Эти данные не должны храниться в selfmanager — сервис selfmanager владеет данными о производительности, а не об обогащении личности пользователя — и не должны храниться в социальной службе, которая владеет социальными взаимодействиями, а не данными профиля.

Сервис `psocial_user` будет выделенным владельцем данных профиля пользователя. Это покажет:

- ПОЛУЧИТЬ `/api/v1/users/me` — собственный профиль аутентифицированного пользователя
- ИСПРАВЛЕНИЕ `/api/v1 / users /me` — обновите поля профиля (отображаемое имя, биографию, фотографию)
- ПОЛУЧИТЬ `/api/v1/users/{id}` — публичный просмотр профиля другого пользователя
- POST `/api/v1/users/{id}/follow` — подписываться на пользователя
- УДАЛИТЬ `/api/v1/users/{id}/подписаться` — отменить подписку
- ПОЛУЧИТЬ `/api/v1/users/me / followers / /` запросы в социальной сети — `following graph`

Сервис будет использовать тот же секрет JWT, что и другие сервисы, и будет доступен социальной службе для получения данных профиля и службе аналитики для включения метаданных профиля в анализ.

3.8.5 Проприетарные модели ML: стратегия накопления данных

Текущий конвейер анализа искусственного интеллекта использует большие языковые модели для всего анализа — преднамеренный выбор для ранней стадии платформы, когда нет данных о поведении пользователей для обучения. По мере того, как платформа накапливает данные о реальном использовании, параллельная разработка собственной прогностической модели становится жизнеспособной.

Планируемый переход - это не замена анализа на основе LLM, а дополнение. Определенные задачи прогнозирования, к которым LLM подходят качественно (“ваша привычка сопряжена с риском”), могут быть решены более точно обученными моделями, которые извлекли уроки из тысяч реальных пользовательских траекторий:

Прогнозирующая задача	Тип модели	Обучающий сигнал
Вероятность сохранения привычки	Логистическая регрессия / повышение градиента	Журналы выполнения с историей серий
Вероятность завершения задачи при текущей нагрузке	Повышение градиента	Журналы выполнения задач с контекстными функциями

Прогнозирующая задача	Тип модели	Обучающий сигнал
Оптимальная продолжительность сеанса фокусировки для данного пользователя	Регрессия	Завершенный сеанс в сравнении с историей заброшенных сеансов
Ежедневный показатель производительности	Нейронная сеть	Совокупность сигналов задачи, привычки и сосредоточенности
Риск отказа во время сеанса	Повышение градиента	Количество пауз и продолжительность сеанса при завершении

Эти модели будут обучены извне и загружены в реестр моделей биллинговой службы в виде файлов .pkl через существующую конечную точку POST /api/v1/models/{id}/upload. Способ выводаasklearn биллинговой службы — уже реализованный и готовый к приему моделей — будет загружать их через joblib и вызывать их вместе со структурированными данными об активности пользователя в качестве функций. Остальная часть системы (аналитика, мобильное приложение, информационные панели) не требует изменений; модель отображается в окне выбора модели рядом с параметрами LLM.

3.9 Веб-панели мониторинга

ProductiveSocial предоставляет две веб-панели на основе Streamlit: панель администратора (psocial_dashboard, порт 1230) для системных операторов и панель пользователя (psocial_user_dashboard, порт 1231) для конечных пользователей. Оба приложения на Python созданы с помощью Streamlit и взаимодействуют с серверными службами исключительно через те же HTTP API, которые использует мобильный клиент — это настоящие клиенты API, а не специальный внутренний интерфейс.

Почему были созданы информационные панели вместо собственных интерфейсов. Мобильное приложение КМР предназначено для долгосрочного использования как пользователями, так и администраторами, но на текущем этапе разработки оно еще не является полнофункциональным или доступным для пользователей - большинство его экранов все еще находятся в стадии разработки. Создание полнофункционального собственного интерфейса администратора в КМР параллельно с остальной платформой значительно задержало бы разработку внутренних сервисов, которые являются ядром ценностного предложения проекта.

Таким образом, две панели управления Streamlit - это явно краткосрочные решения, призванные сократить разрыв: они делают платформу удобной и наглядной уже сейчас, в то время как мобильное приложение продолжает разрабатываться. Streamlit был выбран специально потому, что он позволяет написать функциональный многостраничный веб-интерфейс на нескольких сотнях строк Python без каких—либо накладных расходов на разработку внешнего интерфейса - один разработчик может создать работающий интерфейс администратора и пользователя за долю времени, которое потребовалось бы для создания эквивалентных собственных экранов. Как только приложение КМР будет полнофункциональным, информационные панели будут заменены мобильным приложением в качестве основного пользовательского интерфейса, хотя панель администратора может продолжать служить инструментом мониторинга и операций.

Такой выбор дизайна имеет два важных следствия: панели мониторинга обеспечивают непрерывное покрытие интеграционными тестами каждой вызываемой ими конечной точки API и демонстрируют, что поверхности API системы достаточно для всех вариантов использования без каких-либо привилегированных обходов (за единственным исключением прямых вызовов выставления счетов панели администратора, которые используют X-Internal-Key как предназначенные для доверенных клиентов уровня обслуживания).

3.10 Развертывание

Локальная разработка с помощью Docker Compose. Все службы и базы данных выполняются в контейнерах Docker в общей мостовой сети. Смотрите Раздел 3.12 для получения полной информации о Docker и Colima.

3.11 Тестирование

Ручное тестирование API. Все конечные точки сервиса были протестированы вручную с использованием HTTP-клиентов (Postman и curl) во время разработки. Подход к тестированию заключался в индивидуальной проверке как правильного пути, так и ключевых ошибок (недопустимый JWT, недостаточное количество кредитов, отсутствующие обязательные поля, неизвестные идентификаторы объектов).

Сквозное тестирование трубопровода. Наиболее важным тестированием был полный конвейер сквозного анализа: создайте тестовые данные в selfmanager → создайте сеансы Pomodoro в timer → вызовите аналитику из панели управления пользователями → проверьте, что выходные данные LLM ссылаются на конкретные пользовательские данные. Этот тест был успешно запущен как локально (через Docker Compose), так и в рабочей среде (при рендеринге с помощью туннеляngrokOllama).....

Проверка идемпотентности синхронизации. Конечная точка пакетной синхронизации была протестирована путем повторной отправки одного и того же запроса на синхронизацию и проверки того, что не было создано повторяющихся записей. Тест пройден для всех типов сущностей.

Проверка арифметической кредитоспособности. Для каждого прогноза в тестовом наборе баланс до прогнозирования, стоимость прогнозирования и баланс после прогнозирования были проверены вручную на соответствие $\text{balance_before} - \text{cost} = \text{balance_after}$.

Что не тестировалось. Нагрузочное тестирование, тестирование производительности при параллельных запросах и тестирование хаоса (имитирующее частичные сбои в обслуживании под нагрузкой) не проводились. Это стандартная практика для производственных систем, и она будет проводиться до любого публичного выпуска.

3.12 Интеграция сервисов: сквозные потоки запросов

В этом разделе прослеживается полный путь выполнения двух наиболее важных операций в системе, связанных с пользователем, - аутентификации и анализа с

помощью искусственного интеллекта — от запроса клиента до окончательного ответа, точно показывая, как участвует каждая служба.

Поток 1: Проведение анализа с использованием искусственного интеллекта (полный сквозной анализ)

Это центральная функция платформы. Она включает в себя все четыре службы, два межведомственных вызова для сбора данных и вывод LLM. Каждый шаг прослеживается полностью.

Общая сквозная задержка для этого потока на уровне без рендеринга с "горячими" сервисами составляет примерно 15-30 секунд, в основном на этапе вывода Ollama. С облачным провайдером LLM (Anthropic или OpenAI) этот процесс сократился бы примерно до 5-10 секунд.

Поток 2: Запись времени по таймеру (межведомственная запись)

Когда рабочий интервал Pomodoro завершен и был привязан к задаче, таймер автоматически обновляет накопленное время задачи в selfmanager:

Шаблон "включи и забудь" означает, что ответ таймера пользователю не блокируется вызовом журнала времени для selfmanager. Если selfmanager временно недоступен, обновление журнала времени теряется - компромисс, принятый для проверки концепции, заключается в том, что во время простоя selfmanager время фокусировки может быть немного занижено, что не приведет к сбою основных функций таймера.

3.14 Соображения безопасности

В этом разделе описываются свойства безопасности текущей реализации и улучшения безопасности, которые будут необходимы перед выпуском для всеобщего использования.

Текущая ситуация в области безопасности

Границы аутентификации устанавливаются на уровне HTTP. Для каждой внутренней конечной точки требуется заголовок X-Internal-Key. Для каждой пользовательской конечной точки (кроме /api/v1/auth/identify и /api/v1/models) требуется действительный JWT. Не прошедшие проверку подлинности запросы получают 401 ответ без какой-либо информации о том, что делает конечная точка.

Секреты хранятся в переменных окружения, а не в коде. Секрет JWT, внутренний ключ API, пароли базы данных и ключи API LLM-провайдера загружаются из переменных среды при запуске. Они никогда не передаются в git, никогда не возвращаются в ответах API и никогда не регистрируются.

Служба выставления счетов не имеет общедоступных конечных точек. Даже если URL-адрес биллинговой службы был каким-либо образом раскрыт (например, через сетевую вкладку браузера), ни одна из ее конечных точек не отвечает на запросы без заголовка X-Internal-Key. Это исключает возможность прямого манипулирования кредитами со стороны пользователей.

Срок действия JWT принудительно истек. Токены доступа истекают через 24 часа. Украденный токен имеет ограниченное окно неправомерного использования.

Ограничения безопасности при проверке концепции

Аутентификация только по электронной почте не является стандартной. Текущий механизм аутентификации принимает любой адрес электронной почты без проверки. В рабочей среде для предотвращения олицетворения учетной записи потребуется проверка электронной почты (или поставщика OAuth2, такого как Google).

Общий секрет JWT - это единая точка доверия. Если секрет JWT скомпрометирован, все службы будут скомпрометированы одновременно. В производстве смена секрета требует одновременного развертывания всех трех пользовательских сервисов. Более надежный подход заключался бы в использовании асимметричного JWT (RS256) с парой открытых / закрытых ключей, позволяющей хранить секретный ключ подписи только службе выдачи.

Нет ограничений по скорости аутентификации. Конечная точка `/api/v1 /auth / identify` не имеет ограничений по скорости, что делает ее уязвимой для атак с перечислением (определение того, на какие адреса электронной почты есть учетные записи). При производстве потребуется ограничение скорости и CAPTCHA.

В локальной разработке нет принудительного использования HTTPS. Локальный Docker Compose работает по HTTP. В рабочей среде на Render все сервисы обслуживаются по протоколу HTTPS обратным прокси-сервером Render. Однако сами службы не поддерживают протокол HTTPS, поэтому при развертывании за прокси-сервером, отличным от HTTPS, данные будут передаваться открытым текстом.

The ngrok для ollama небезопасен для многопользовательского развертывания. URL-адрес ngrok является общим в переменной среды службы выставления счетов. Любая служба с правильным X-Internal-Key может выполнять вызовы вывода через этот туннель, и весь трафик вывода проходит через инфраструктуру ngrok. Для производственного развертывания выделенная облачная модель, обслуживающая конечную точку (с надлежащей аутентификацией и ограничением скорости), заменила бы схему ngrok.

3.15 Окончательная проверка

Окончательная проверка концепции ProductiveSocial направлена на решение четырех конкретных проблем, поднятых руководителем:

1. Поддающаяся проверке валидация и базовый уровень (рассматриваются в разделе 2.7)

Таблица экспериментов в разделе 2.7 документирует 12 конкретных тестов с четкими базовыми показателями, метриками, результатами и заключениями. Наиболее значимая проверка — качество анализов, сгенерированных LLM, по сравнению с базовым уровнем, основанным на правилах, - демонстрирует явное и существенное улучшение: баллы LLM составляют 3.9–4.2/5 по сравнению с базовыми баллами 1.0–1.5/5 по трем типам анализа.

2. Названные конкуренты и сравнительный анализ (рассмотрены в разделе 1.3)

Были проанализированы семь конкретных конкурирующих приложений по шести параметрам, в результате чего была составлена сравнительная таблица, подтверждающая утверждение о новизне. Уникальное сочетание четырех модулей повышения производительности с междоменным уровнем анализа искусственного интеллекта отсутствует ни у одного из проанализированных конкурентов.

3. Описание архитектуры микросервисов (рассмотрено в разделах 3.2, 3.3, 3.4)

Подробно описана архитектура: сервисы с явными описаниями модели данных, протоколы межсервисной связи (JWT для связи клиент-сервис, X-Internal-Key для связи сервис-сервис) и UserRegistry механизм межсервисной идентификации. Схема взаимодействия службы выглядит следующим образом:

```
psocial_user_dashboard — JWT → psocial_analytics
                        — JWT → psocial_selfmanager
                        — JWT → psocial_timer

psocial_admin_dashboard — X-Internal-Key → psocial_billing
                        — JWT → psocial_analytics

psocial_analytics — X-Internal-Key → psocial_selfmanager
                  — X-Internal-Key → psocial_timer
                  — X-Internal-Key → psocial_billing

psocial_timer — X-Internal-Key → psocial_selfmanager (time-log, fire-and-forget)

psocial_timer — JDBC → selfmanager_db (users table – UserRegistry)
```

Отказоустойчивость достигается за счет: неблокирующих исходящих вызовов в службе аналитики (сбои в работе selfmanager и таймера возвращают пустые данные, а не отказывают в выполнении запроса), аварийных вызовов журнала времени от таймера (доступность таймера не блокируется selfmanager), независимых баз данных для каждой службы (отсутствие общего состояния, которое могло бы вызвать каскадные сбои) и независимого развертывания на уровне службы (каждая служба может быть перезапущена, не затрагивая другие).

4. Комплексный сценарий ML / LLM (рассматривается в разделах 2.4, 2.5, 2.7)

Задokumentирован полный сквозной сценарий: входные данные → построение запроса → Вызов LLM → извлечение выходных данных → кредитный вычет → сохранение отчета → отображение пользователем. Фактическая структура запроса, фактический результат LLM для PRODUCTIVITY_SUMMARY и сравнение с базовым уровнем, основанным на правилах, представлены в разделе 2.7.

Заключение

В этом отчете задокументирован полный жизненный цикл первого proof-of-concept от ProductiveSocial: от определения предметной области и структурированного анализа конкурентной среды, архитектурного проектирования и технической реализации четырех независимых микросервисов до комплексной проверки основного ценностного предложения системы.

Краткое изложение материалов

Определение предметной области и проблемы. В отчете был выявлен и задокументирован структурный пробел на рынке инструментов для повышения личной продуктивности: ни одно существующее приложение не предоставляет все четыре основных модуля повышения продуктивности (управление задачами, отслеживание привычек, хронометраж Pomodoro и анализ с помощью искусственного интеллекта) на единой платформе, и ни одно существующее приложение не предоставляет уровень искусственного интеллекта с одновременным доступом к междоменным поведенческим данным. Этот разрыв был подтвержден сравнительным анализом семи ведущих продуктов — Todoist, Habitica, Forest, Notion, Day One, RescueTime и Sunsama — каждый из которых был проанализирован по шести параметрам возможностей.

Архитектурный проект. Архитектура ProductiveSocial была разработана на основе пяти четких принципов — независимости сервиса, единого источника достоверности для каждого домена, границ аутентифицированного доступа, владения данными в автономном режиме и системы кредитов как поведенческого стимула. Эти принципы привели к конкретным, оправданным проектными решениям: JDBC-соединение UserRegistry с кросс-базой данных для канонической идентификации пользователя; модель журнала транзакций в качестве баланса биллинговой службы для доказуемо согласованного учета кредитоспособности; неблокирующий сбор данных восходящего потока службой аналитики для постепенного ухудшения; и общий секрет JWT для межведомственной аутентификации без централизованного поставщика идентификационных данных.

Техническая реализация. Реализация охватывает четыре серверных сервиса (Kotlin /Ktor для selfmanager, timer и analytics; Python /FastAPI для выставления счетов), две веб-панели мониторинга с потоковым управлением (для администратора и пользователя) и мультиплатформенное мобильное приложение Kotlin на основе каркасов. Выбор технологий — Kotlin, Ktor, FastAPI, PostgreSQL, Docker, Colima, Streamlit, KMP, SQLDelight — был сделан по конкретным техническим причинам, описанным в разделе 2.2, с общей

целью использовать лучший инструмент для каждой задачи, а не навязывать один язык или фреймворк всей системе.

Интеграция ML. Конвейер анализа ИИ реализует шаблон генерации с дополнением к поиску (RAG), в котором данные о пользователях собираются во время вывода, а не из векторной базы данных. Были разработаны и протестированы шесть типов анализа с различными подсказками. Валидация по базовому уровню, основанному на правилах, продемонстрировала существенное улучшение качества: баллы LLM составили 3.9–4.2/5 по сравнению с базовыми баллами 1.0–1.5/5 по трем типам анализа. Анализ выявил междоменные поведенческие корреляции и дал конкретные рекомендации, которые можно реализовать - возможности, которые определенно выходят за рамки любого подхода, основанного на правилах.

Проверка. Двенадцать документированных экспериментов подтвердили функциональную корректность (межсервисная аутентификация JWT, арифметика зачета, идемпотентность синхронизации, постепенное ухудшение), качество LLM (три сравнения типов анализа с примерами входных и выходных данных) и операционное поведение (задержка холодного запуска, задержка прогрева, исключение зачета при сбое прогнозирования).

Текущие ограничения

Проверка концепции демонстрирует основное ценностное предложение в полностью функциональной системе, но имеет ряд ограничений, которые отличают ее от готовой к производству платформы:

- **Комплектность мобильного приложения.** Приложение KMP построено с использованием схемы потока аутентификации и SQLDelight, но экраны задач, привычек, рутины, таймера и аналитики еще не реализованы. Основной пользовательский интерфейс для конечных пользователей остается незавершенным.
- **Хостинг для юридических лиц.** В производственном развертывании используется личный экземпляр, который предоставляется через ngrok. Это не масштабируемо (компьютер одного разработчика), ненадежно (компьютер должен быть запущен и подключен) и не подходит для многопользовательской системы. Для обеспечения готовности к производству требуется переключение на облачного провайдера LLM (Anthropic Claude или OpenAI GPT-4o) или на развертывание Ollama в облаке.
- **Уровень развертывания.** Бесплатный уровень Render предусматривает 50-секундный "холодный запуск" служб JVM после 15 минут простоя.

Платный уровень развертывания с постоянными контейнерами устранил бы эту проблему взаимодействия с пользователем.

- **Модель аутентификации.** Аутентификация "найти или создать" только по электронной почте подходит для проверки концепции, но не для системы с реальными пользовательскими данными. Потребуется проверка электронной почты, интеграция с поставщиком OAuth2 или аутентификация на основе пароля.

Дорожная карта: следующий этап разработки

На следующем этапе разработки устраняются вышеуказанные ограничения в приоритетном порядке:

Фаза 2 (немедленная):

1. Дополните экраны мобильного приложения КМР информацией о задачах, привычках, распорядке дня и таймере Pomodoro
2. Настройте Anthropic Claude в качестве производственного LLM-провайдера в службе выставления счетов
3. Обновите развертывание рендеринга, чтобы исключить "холодные запуски"
4. Реализовать проверку электронной почты в потоке аутентификации

Этап 3 (среднесрочный):

5. Реализовать социальный модуль в виде нового микросервиса (общедоступные профили, обмен достижениями, лента сообщества)
6. Добавьте модуль журнала с обобщением дня с помощью искусственного интеллекта
7. Внедрите push-уведомления для напоминаний о привычках и времени начала рутинной работы
8. Добавьте модуль notes с перекрестными ссылками на объекты

Фаза 4 (долгосрочная):

9. Накапливайте реальные данные о поведении пользователей и приступайте к обучению проприетарных прогностических моделей
10. Внедряйте обученные модели sklearn с помощью существующего пути вывода биллинговой службы

11. Построение модели устройства для полностью автономных функций искусственного интеллекта
12. Сторонние интеграции (Календарь Google, носимые устройства)

Архитектурная готовность к росту

Архитектура специально разработана для размещения дополнений фазы 3 и фазы 4, не требуя структурных изменений в существующих сервисах. Новые сервисы (социальные сети, журналы, заметки) могут быть добавлены как независимые микросервисы, которые используют одну и ту же модель идентификации пользователя (selfmanager целочисленные идентификаторы пользователей) и один и тот же механизм аутентификации (общий секрет JWT или X-Internal-Key). Реестр моделей биллинговой службы и путь вывода sklearn уже созданы для получения обученных моделей, когда они будут готовы. Система подсказок службы аналитики может быть расширена новыми типами анализа путем добавления нового обращения к функции buildPrompt() и нового постоянного типа анализа — никаких других изменений службы не требуется.

ProductiveSocial - это, по своей сути, платформа, созданная как для того, чем она станет, так и для того, чем она уже является. Решения, принятые на этом первом этапе — общая модель идентификации пользователей, внутренние границы API, кредитная книга на основе транзакций, протокол синхронизации в автономном режиме - не являются вариантами реализации, которые необходимо будет пересматривать по мере роста платформы. Они являются архитектурным фундаментом, на котором может быть построено полное видение интеллектуальной операционной системы для повышения производительности.

Список литературы

1. Allen, D. (2001). Доводить дело до конца: искусство продуктивности без стресса. Viking.
2. Чирилло, Ф. (2006). Техника Помодоро. ФК "Гараж".
3. Ньюпорт, С. (2016). Глубокая работа: правила целенаправленного успеха в отвлеченном мире. Издательство Grand Central Publishing.
4. Клир, Дж. (2018). Атомарные привычки: простой и проверенный способ выработать хорошие привычки и избавиться от плохих. Эйвери.
5. Лерой, С. (2009). Почему мне так трудно выполнять свою работу? Проблема остатков внимания при переключении между рабочими задачами. Организационное поведение и процессы принятия решений человеком, 109 (2), 168-181.
6. Ньюман, С. (2021). Создание микросервисов: проектирование мелкозернистых систем (2-е изд.). О'Рейли Медиа.
7. Ричардсон, С. (2018). Шаблоны микросервисов: с примерами на Java. Публикации Мэннинга.
8. Льюис Дж., & Фаулер М. (2014). Микросервисы: определение этого нового архитектурного термина. [martinfowler.com. https://martinfowler.com/articles/microservices.html](https://martinfowler.com/articles/microservices.html)
9. Браун Т., Манн Б., Райдер Н. и др. (2020). Языковые модели Изучают Немногие. Достижения в области нейронных систем обработки информации, 33, 1877-1901.
10. Васвани А., Шазир, Н., Пармар Н. и др. (2017). Внимание - это все, что вам нужно. Достижения в области нейронных систем обработки информации, 30.
11. Команда разработчиков задач. (2024). Todoist: Список дел для организации работы и жизни. <https://todoist.com>
12. Команда Habitica. (2024). Habitica: Геймифицируйте свою жизнь. <https://habitica.com>
13. Notion Labs, Inc. (2024). Notion: подключенное рабочее пространство для ваших заметок, документов и проектов. <https://notion.so>

14. Команда первого дня. (2024). День первый: приложение для ведения журнала. <https://dayoneapp.com>
15. Команда времени спасения. (2024). RescueTime: программное обеспечение для управления временем. <https://rescuetime.com>
16. Touchlight Ltd. (2024). Сансама: Ежедневник. <https://sunsama.com>
17. Фонд Котлина. (2024). Мультиплатформенный Kotlin. <https://kotlinlang.org/docs/multiplatform.html>
18. Реактивные мозги. (2024). Ktor: Фреймворк для создания асинхронных серверных приложений. <https://ktor.io>
19. FastAPI. (2024). FastAPI: современный, быстрый веб-фреймворк для создания API на Python. <https://fastapi.tiangolo.com>
20. Оллама. (2024). Оллама: Освоение больших языковых моделей. <https://ollama.ai>
21. Антропный. (2024). Клод: Ассистент по искусственному интеллекту от Anthropic. <https://anthropic.com>
22. SQLDelight. (2024). SQLDelight: Типизированный SQL для мультиплатформенной Kotlin. [https://cashapp.github.io/sqldelight /](https://cashapp.github.io/sqldelight/)
23. Визуализировать. (2024). Визуализация: самый быстрый способ размещения всех ваших веб-приложений. <https://render.com>
24. Neon. (2024). Neon: Бессерверная Postgres. <https://neon.tech>
25. Фаулер, М. (2002). Шаблоны архитектуры корпоративных приложений. Эддисон-Уэсли.
26. Деси, Э. Л. и Райан, Р. М. (1985). Внутренняя мотивация и самоопределение в поведении человека. Спрингер.
27. Свеллер, Дж. (1988). Когнитивная нагрузка при решении проблем: влияние на обучение. Когнитивная наука, 12(2), 257-285.
28. Чирилло, Ф. (2018). Техника Помодоро: признанная система тайм-менеджмента, которая изменила то, как мы работаем. Валюта.
29. Реактивные мозги. (2024). Обзор мультиплатформенной платформы Kotlin. <https://kotlinlang.org/docs/multiplatform.html>
30. Реактивные мозги. (2024). Документация Ktor: асинхронный фреймворк Kotlin. [https://ktor.io/docs /](https://ktor.io/docs/)

31. FastAPI. (2024). Документация по FastAPI. <https://fastapi.tiangolo.com/>
32. Docker Inc. (2024). Документация по Docker: начало работы. <https://docs.docker.com/>
33. Колима. (2024). Colima: среды выполнения контейнеров в macOS (и Linux) с минимальной настройкой. <https://github.com/abiosoft/colima>
34. SQLDelight. (2024). SQLDelight: Типизированный SQL для мультиплатформенной Kotlin. <https://cashapp.github.io/sqldelight/>
35. Тианголо, С. (2024). Документация Pydantic. <https://docs.pydantic.dev/>
36. Джонс, М., Брэдли, Дж., & Сакимура, Н. (2015). RFC 7519: веб-токен JSON (JWT). IETF. <https://datatracker.ietf.org/doc/html/rfc7519>
37. Атласский. (2024). Потокковая документация. <https://docs.streamlit.io/>
38. Meta AI. (2024). Llama 3.2: Открытая основа и доработанные модели чатов. <https://ai.meta.com/>

Приложение А: Глоссарий технических терминов

API (интерфейс прикладного программирования): определенный набор правил и конечных точек, с помощью которых одна программная система взаимодействует с другой. В контексте этого проекта каждая служба предоставляет HTTP API, который клиенты и другие службы вызывают для создания, чтения, обновления или удаления данных.

Асинхронное программирование: модель программирования, в которой иницируются операции, которые в противном случае были бы заблокированы (ожидание ответа сети, результата базы данных, таймера), а затем программа продолжает выполнять другую работу; когда операция завершается, обратный вызов или продолжение возобновляют обработку. Сопрограммы Kotlin реализуют эту модель.

Кодировка Base64url: вариант кодировки Base64, который безопасен для использования в URL-адресах и заголовках HTTP (замена + на - и / на _). Используется для кодирования разделов заголовка и полезной нагрузки JWT.

CORS (совместное использование ресурсов разных источников): Механизм HTTP, который контролирует, каким веб-источникам разрешено отправлять запросы к серверу. Все службы Ktor настроены так, чтобы разрешать запросы от источников панели мониторинга.

CRUD: Создание, чтение, обновление, Удаление — четыре основные операции с постоянными данными, соответствующие HTTP POST, GET, PUT/PATCH и DELETE соответственно.

DTO (Объект передачи данных): простой класс данных, используемый для передачи данных между процессами. В службах Kotlin DTO определяются как классы данных, которые сопоставляются с телами запросов / ответов JSON.

HMAC (код аутентификации сообщения на основе хэша): криптографическая конструкция, которая объединяет хэш-функцию (SHA-256 в случае HS256 JWTs) с секретным ключом для создания кода аутентификации сообщения. Используется в качестве алгоритма подписи JWT.

HTTP-заголовок: Пара ключ-значение, включенная в HTTP-запрос или ответ для передачи метаданных. Заголовок авторизации содержит токен-носитель JWT; заголовок X-Internal-Key содержит ключ аутентификации между сервисами; Content-Type определяет формат тела запроса.

Идемпотентность: свойство операции, при котором многократное ее повторение приводит к тому же результату, что и при ее однократном выполнении. Конечная точка синхронизации является идемпотентной: при повторной отправке одного и того же запроса синхронизации одни и те же

записи создаются один раз, а не дважды. Это важно для безопасности повторных попыток.

JDBC (Java Database Connectivity): Java API для подключения к реляционным базам данных. Службы Kotlin используют JDBC (с HikariCP для объединения в пул) для подключения к PostgreSQL. UserRegistry использует исходное соединение JDBC, а не ORM, поскольку оно выполняет единственную конкретную инструкцию SQL, которая выигрывает от явного контроля.

JVM (виртуальная машина Java): Среда выполнения для программ на Java и Kotlin. Скомпилированный класс Kotlin выполняется в JVM, который обеспечивает сборку мусора, JIT-компиляцию и независимость от платформы. Время запуска JVM (~ 2-3 секунды) является причиной того, что службы на основе JVM имеют более длительные холодные запуски, чем служба выставления счетов Python.

JWT (веб-токен JSON): компактный, безопасный для URL-адресов формат для представления утверждений в виде объекта JSON, подписанный HMAC или RSA. Полное описание смотрите в разделе 2.2.6.

KMP (мультиплатформенный Kotlin): технология совместного использования кода Kotlin на нескольких платформах (JVM, iOS, JavaScript, native). Полное описание смотрите в разделе 2.2.8.

LLM (Large Language Model): нейросетевая модель, обученная на больших объемах текстовых данных, которая может генерировать связные, контекстуально соответствующие текстовые ответы на запросы. Примеры: Антропный Клод, OpenAI GPT-4o, Meta Llama. Используется в ProductiveSocial через механизм вывода биллинговой службы.

Микросервис: Независимо развертываемый программный сервис с единой, целенаправленной ответственностью. Полное описание смотрите в разделе 2.2.1.

ORM (объектно-реляционный картограф): Библиотека, которая выполняет перевод между объектно-ориентированными представлениями, используемыми в коде приложения, и реляционными представлениями, используемыми в базах данных SQL. Exposed (используется в сервисах Kotlin) и SQLAlchemy (используется в службе выставления счетов) являются ORM в этом проекте.

Разработка подсказок: Практика разработки текстовых вводов (подсказок) для получения определенных типов ответов из языковой модели. Смотрите Раздел 2.5 для получения полного описания запросов, зависящих от типа анализа, используемых в этом проекте.

RAG (поисково-расширенная генерация): шаблон для приложений на основе LLM, в котором релевантные данные извлекаются из источника данных и

включаются в запрос LLM во время вывода, предоставляя модели текущий конкретный контекст, который она не могла бы усвоить только из предварительной подготовки. Аналитический конвейер ProductiveSocial отявляется реализацией этого шаблона.

REST (передача репрезентативного состояния): Архитектурный стиль для HTTP API, характеризующийся парами запросов / ответов без сохранения состояния, URL-адресами, ориентированными на ресурсы, и стандартными HTTP-глаголами (GET, POST, PUT, DELETE). Все службы ProductiveSocial предоставляют REST API.

Синглтон: Шаблон проектирования, в котором класс имеет не более одного экземпляра в приложении, общего для всего кода, который зависит от него. Одиночное объявление `Koin { }` создает синглтоны для всех сервисов и HTTP-клиентов.

Надгробный камень: Запись, отмечающая удаление другой записи, используемая в протоколах синхронизации для сообщения о том, что запись была удалена на сервере и должна быть удалена из локальной базы данных клиента.

UUID (универсально уникальный идентификатор): 128-битный идентификатор, сгенерированный таким образом, что коллизии практически невозможны. Используется в качестве идентификаторов, назначаемых клиентом для всех синхронизированных объектов, что позволяет создавать автономные записи без координации с сервером.

ПРИЛОЖЕНИЯ